

UNIVERZITA PARDUBICE

FAKULTA ELEKTROTECHNIKY A
INFORMATIKY

BAKALÁŘSKÁ PRÁCE

2026

Egor Razboinikov

Univerzita Pardubice
Fakulta Elektrotechniky a Informatiky

Digitalizace evidence pracovní doby

Bakalářská práce

Univerzita Pardubice
Fakulta elektrotechniky a informatiky
Akademický rok: 2024/2025

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení:	Egor Razboinikov
Osobní číslo:	I21219
Studijní program:	B0688A140009 Informační technologie
Téma práce:	Digitalizace evidence pracovní doby
Zadávací katedra:	Katedra informačních technologií

Zásady pro vypracování

Dokument *evidence pracovní doby* vyučujícího vyžaduje provázanost rozvrhu s úvazkem vyučujícího při respektování času přestávek. Cílem bakalářské práce je navrhnout, vytvořit, odzkoušet a předat funkční multiplatformní aplikaci jejíž vstupy jsou: rozvrh vyučujícího, čas na oběd, plán dovolených, státní svátky, školní volna a další. Požadovaným výstupem je PDF dokument (tabulka) se všemi náležitostmi potřebnými pro odevzdání. Grafické rozhraní poskytne pohodlné zadávání, umožní dodatečné ruční modifikace a zobrazování náhledu před vygenerováním PDF dokumentu.

Rozsah pracovní zprávy: **min. 30 stran**
Rozsah grafických prací:
Forma zpracování bakalářské práce: **tištěná**

Seznam doporučené literatury:

PŠENČÍKOVÁ, J. Algoritmizace. Kralice na Hané, Computer Media, 2009.
MAREŠ, Martin a VALLA, Tomáš. Průvodce labyrintem algoritmů 2. vydání. Praha: CZ.NIC, 2022. ISBN 978-80-88168-63-8
PECINOVSKÝ, Rudolf. OOP: naučte se myslet a programovat objektově. Brno: Computer Press, 2010. ISBN 978-80-251-2126-9.

Vedoucí bakalářské práce: **Ing. Radek Matoušek**
Katedra informačních technologií

Datum zadání bakalářské práce: **15. prosince 2024**
Termín odevzdání bakalářské práce: **16. května 2025**

prof. Ing. Petr Doležel, Ph.D. v.r.
děkan

L.S.

Ing. Jan Panuš, Ph.D. v.r.
vedoucí katedry

Prohlašuji:

Práci s názvem Digitalizace evidence pracovní doby jsem vypracoval samostatně. Veškeré literární prameny a informace, které jsem v práci využil, jsou uvedeny v seznamu použité literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 7/2019 Pravidla pro odevzdávání, zveřejňování a formální úpravu závěrečných prací, ve znění pozdějších dodatků, bude práce zveřejněna prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 9. 5. 2026

Egor Razboinikov v.r.

PODĚKOVÁNÍ

Rád bych poděkoval vedoucímu své bakalářské práce Ing. Radku Matouškovi za odborné vedení, cenné připomínky a ochotu při konzultacích během zpracování této práce. Poděkování rovněž patří mé rodině a přátelům za podporu po celou dobu studia.

ANOTACE

Cílem této bakalářské práce je navrhnout, implementovat a ověřit multiplatformní desktopovou aplikaci pro digitalizaci evidence pracovní doby vyučujících. Aplikace umožňuje import rozvrhových dat ze systému STAG, jejich další zpracování a úpravu v grafickém uživatelském rozhraní. Při zpracování zohledňuje polední přestávky, plánované nepřítomnosti, státní svátky a pravidla evidence pracovní doby. Výstupem aplikace je přehledný PDF dokument odpovídající požadavkům na odevzdání. Práce se zabývá analýzou problematiky, návrhem architektury a technologií, implementací klíčových částí systému a ověřením výsledného řešení pomocí testování.

KLÍČOVÁ SLOVA

evidence pracovní doby, digitalizace, aplikace pro osobní počítače, C#, .NET, Avalonia UI, MVVM, SQLite, Entity Framework Core, STAG, generování PDF, testování aplikace.

TITLE

Digitization of Working Time Records

ANNOTATION

The aim of this bachelor thesis is to design, implement, and validate a cross-platform desktop application for digitizing teachers' working time records. The application enables the import of timetable data from the STAG system, their further processing, and manual adjustment through a graphical user interface. During processing, it takes into account lunch breaks, planned absences, public holidays, and the rules of working time records. The output of the application is a structured PDF document corresponding to submission requirements. The thesis deals with the analysis of the problem domain, the design of the architecture and technologies, the implementation of key system components, and the validation of the resulting solution through testing.

KEYWORDS

working time records, digitization, desktop application, C#, .NET, Avalonia UI, MVVM, SQLite, Entity Framework Core, STAG, PDF generation, application testing.

OBSAH

SEZNAM ILUSTRACÍ.....	11
SEZNAM ZKRATEK A ZNAČEK.....	12
TERMINOLOGIE	13
ÚVOD.....	14
1 TEORETICKÁ ČÁST	15
1.1 Evidence pracovní doby	15
1.2 Analýza současných přístupů k evidenci pracovní doby	16
1.3 Požadavky na softwarové řešení.....	17
1.3.1 Funkční požadavky	17
1.3.2 Kvalitativní požadavky	18
1.3.3 Shrnutí požadavků	19
1.4 Softwarová architektura a návrhové vzory	20
1.4.1 Softwarová architektura	20
1.4.2 Návrhový vzor MVVM	20
1.4.3 Srovnání MVVM a MVC	22
1.4.4 Monolitická, modulární a mikroslužbová architektura.....	22
1.4.5 Zdůvodnění zvoleného řešení	23
1.5 Použité technologie.....	24
1.5.1 Platforma .NET a jazyk C#	24
1.5.2 Databázová vrstva: SQLite a jazyk SQL	25
1.5.3 Framework Avalonia UI.....	26
1.5.4 Podpůrné knihovny	26
1.5.5 Shrnutí použitých technologií.....	27
1.6 Vývojové prostředí a podpůrné nástroje.....	27
1.6.1 Microsoft Visual Studio	28
1.6.2 SQLiteStudio	28

1.6.3 Shrnutí vývojového prostředí a podpůrných nástrojů.....	29
2 PRAKTICKÁ ČÁST	30
2.1 Návrh řešení.....	30
2.2 Struktura aplikace	31
2.3 Adresářová struktura projektu	32
2.3.1 Prezentací vrstva.....	32
2.3.2 Datová a aplikační vrstva	33
2.3.3 Kořenové soubory projektu	34
2.4 Struktura databáze	34
2.4.1 Přehled tabulek	35
2.4.2 Shrnutí databázového návrhu	37
2.5 Uživatelské rozhraní aplikace.....	38
2.5.1 Hlavní stránka aplikace	38
2.5.2 Dialogové okno pro vytvoření události	38
2.5.3 Dialogové okno pro změnu události	39
2.5.4 Vizuální prezentace pro den.....	40
2.5.5 Vizuální prezentace pro týden	41
2.5.6 Vizuální prezentace pro měsíc	41
2.5.7 Globální nastavení	41
2.5.8 Ukázka PDF.....	42
2.5.9 Shrnutí kapitoly	42
2.6 Algoritmy.....	42
2.6.1 Algoritmus vyrovnávání hodin	43
2.6.2 Algoritmus importu rozvrhu ze STAG.....	45
2.7 Testování aplikace	46
2.7.1 Cíl a kritéria přijetí.....	46
2.7.2 Testovací prostředí.....	47

2.7.3 Metodika ověřování funkčnosti	47
2.7.4 Předpokládané rozdíly mezi platformami.....	48
2.7.5 Testovací scénáře	48
2.7.6 Souhrnná tabulka ověřených scénářů a řešených stavů.....	49
2.7.7 Shrnutí testování a známá omezení	50
ZÁVĚR.....	51
POUŽITÁ LITERATURA	52

SEZNAM ILUSTRACÍ

Obrázek 1 Vztah mezi částmi View, ViewModel a Model v architektuře MVVM	21
Obrázek 2 Architektura MVC	22
Obrázek 3 Ukázka třídy ApplicationDbContext	33
Obrázek 4 Relační schéma databáze	35
Obrázek 5 Dialogové okno pro vytvoření události	39
Obrázek 6 Dialogové okno pro změnu události.....	40

SEZNAM ZKRATEK A ZNAČEK

EPD – evidence pracovní doby

PDF – Portable Document Format

SQL – Structured Query Language

UI – User Interface

MVVM – Model–View–ViewModel

MVC – Model–View–Controller

ISO/IEC – International Organization for Standardization / International Electrotechnical Commission

TERMINOLOGIE

Avalonia UI – Multiplatformní framework pro tvorbu uživatelských rozhraní v .NET s deklarativním popisem obrazovek v AXAML. Umožňuje jednotný vzhled a chování na Windows, Linuxu i macOS.

Framework – Sada knihoven, nástrojů a doporučených postupů, která dává aplikaci strukturu a opakovaně použitelné stavební bloky.

Relační databáze – Datový model založený na tabulkách a vztazích (vazby 1:1, 1:N, N:M) s důrazem na integritu a efektivní dotazování.

MVVM (Model–View–ViewModel) – Vzor, který odděluje data a logiku (Model), prezentační logiku (ViewModel) a zobrazení (View). Zlepšuje testovatelnost a udržitelnost UI.

Uživatelské rozhraní (UI) – Vizuální a interaktivní část aplikace (okna, prvky, šablony), v této práci vytvořená v Avalonia UI/AXAML.

IDE – Integrované vývojové prostředí (editor, debugger, správce závislostí a verzí) pro efektivní vývoj.

VCS – Systémy pro správu verzí (např. Git) pro sledování změn a týmovou spolupráci.

STAG – Studijní agenda, zdroj rozvrhových dat pro import.

EPD – Evidence pracovní doby, oficiální výkaz s měsíčním a týdenním součtem dle interních pravidel.

ÚVOD

Evidence pracovní doby představuje důležitou součást personální a administrativní agendy v různých typech organizací. Slouží k zaznamenávání odpracovaného času zaměstnanců, kontrole plnění pracovních povinností a jako podklad pro další související procesy. Správné vedení evidence pracovní doby je významné nejen z hlediska pracovněprávních požadavků, ale také z hlediska přehlednosti, správnosti a efektivity administrativního zpracování údajů.

V praxi může být vedení evidence pracovní doby spojeno s řadou obtíží. Ty se týkají zejména ručního zadávání údajů, nejednotnosti používaných nástrojů, zvýšené chybovosti a časové náročnosti při zpracování nebo kontrole záznamů. Tyto problémy se mohou dále prohlubovat zejména v prostředí, kde je pracovní režim méně pravidelný a kde je nutné zohledňovat větší množství vstupních údajů, změn a výjimek.

Cílem této bakalářské práce je navrhnout, implementovat a ověřit softwarové řešení, které usnadní evidenci pracovní doby a přispěje ke zpřehlednění celého procesu. Navržená aplikace má umožnit evidenci a úpravu záznamů pracovní doby, zpracování vstupních údajů a vytvoření přehledného výstupu pro další administrativní využití.

Teoretická část práce se nejprve zabývá vymezením evidence pracovní doby a analýzou současných přístupů k jejímu vedení. Dále jsou formulovány požadavky na navržené softwarové řešení a jsou popsány architektonické a technologické prostředky vhodné pro jeho realizaci.

Praktická část práce je věnována návrhu, implementaci a ověření vytvořené aplikace. Zaměřuje se na strukturu aplikace, databázový model, uživatelské rozhraní, algoritmy pro zpracování dat a testování výsledného řešení. Přínosem práce je návrh a realizace aplikace, která může přispět ke zjednodušení evidence pracovní doby a ke snížení administrativní náročnosti tohoto procesu.

1 TEORETICKÁ ČÁST

1.1 Evidence pracovní doby

Evidence pracovní doby představuje významný nástroj pracovněprávní i administrativní praxe. Jejím účelem je zachycovat průběh pracovní doby zaměstnance tak, aby bylo možné kontrolovat dodržování zákonných limitů, vyhodnocovat rozsah odvedené práce a vytvářet podklady pro navazující personální a mzdové procesy. Povinnost vést evidenci pracovní doby ukládá zaměstnavateli zákoník práce, konkrétně § 96, přičemž tato povinnost slouží zejména ke kontrole pracovní doby, doby odpočinku, práce přesčas a dalších skutečností souvisejících s výkonem práce [1, 2].

Z hlediska právní úpravy musí být evidence vedena alespoň v takovém rozsahu, aby z ní byl patrný začátek a konec odpracované směny, odpracovaná práce přesčas, odpracovaná noční práce, odpracovaná doba v době pracovní pohotovosti a pracovní pohotovost, kterou zaměstnanec držel. V širší organizační praxi však evidence pracovní doby obvykle zahrnuje také další údaje důležité pro celkové vyhodnocení pracovního dne, například interně sledované přestávky, nepřítomnosti nebo jiné skutečnosti, které mají význam pro správné posouzení průběhu práce a pro následné administrativní zpracování [2].

Evidence pracovní doby úzce souvisí také s rozvržením pracovní doby. Zaměstnavatel určuje začátek a konec směn a zajišťuje rozvržení pracovní doby, vůči němuž lze následně porovnávat skutečně odpracovaný čas. Právě rozdíl mezi rozvrženou a skutečně odpracovanou dobou je v praxi podstatný například při posuzování práce přesčas, nedodržení plánovaného rozsahu práce nebo jiných odchylek od předpokládaného pracovního režimu [1, 3]. Z tohoto pohledu evidence neplní pouze roli pasivního záznamu, ale představuje i nástroj průběžné kontroly pracovního režimu.

V prostředí vysokých škol má evidence pracovní doby určitá specifika. Akademická činnost se totiž neskládá pouze z přímé výuky, ale také z činností tvůrčích, výzkumných, administrativních a organizačních. Ze zákonného vymezení role vysokých škol i akademických pracovníků lze dovodit, že pracovní režim v tomto prostředí bývá méně pravidelný než v provozech s pevně stanovenými směnami. Přesná evidence pracovní doby tak může být náročnější nejen z hlediska organizace práce, ale také z hlediska vhodně zvoleného technického řešení [4].

Požadavek na kvalitní evidenci pracovní doby potvrzuje i judikatura Soudního dvora Evropské unie, podle níž mají členské státy zajistit, aby zaměstnavatelé používali objektivní, spolehlivý a přístupný systém umožňující měřit délku denní pracovní doby každého pracovníka. Pro praxi

to znamená, že evidence pracovní doby by neměla být pouze formální povinností, ale měla by být vedena přehledně, konzistentně a způsobem, který umožní její následnou kontrolu i ověření správnosti zaznamenaných údajů [5].

1.2 Analýza současných přístupů k evidenci pracovní doby

Způsob vedení evidence pracovní doby se v praxi mezi jednotlivými organizacemi liší. Přestože právní povinnost sledovat pracovní dobu zůstává stejná, konkrétní forma evidence závisí na charakteru organizace, míře digitalizace, složitosti pracovních režimů i na požadavcích na další zpracování údajů. V obecné rovině lze rozlišit tři základní přístupy: ruční evidenci, evidenci prostřednictvím tabulkových nástrojů a evidenci pomocí specializovaných informačních systémů.

Nejjednodušší formu představuje ruční evidence, například v papírové podobě nebo prostřednictvím jednoduchých formulářů. Její výhodou je minimální technická náročnost a snadná dostupnost i v prostředích, kde nejsou k dispozici specializované nástroje. Z hlediska dlouhodobého používání však tento přístup vykazuje řadu omezení. Ruční zapisování zvyšuje pravděpodobnost chyb, komplikuje následnou kontrolu a znesnadňuje vyhledávání, sdílení i archivaci údajů. Současně hůře naplňuje požadavek na objektivní a spolehlivý systém evidence, který je v současné praxi považován za důležitý zejména z hlediska kontroly a prokazatelnosti údajů [5].

Častým mezistupněm mezi ruční a plně automatizovanou evidencí je využití tabulkových nástrojů. Ty umožňují vytvářet vlastní šablony, provádět základní výpočty a připravovat souhrnné přehledy. Výhodou tohoto řešení je relativní flexibilita a nízká pořizovací náročnost. Současně však zůstává značná část procesu závislá na ručním zadávání údajů a na správném nastavení použitých vzorců. Při vyšší variabilitě pracovního režimu nebo při potřebě pravidelných oprav, kontrol a verzování dat se mohou tabulkové nástroje stávat méně přehlednými a méně odolnými vůči chybám. Tento problém se zvyrazňuje zejména v prostředích, kde se pracovní činnost neřídí jednotným a opakujícím se režimem.

Vyspělejší variantu představují specializované informační systémy určené pro evidenci pracovní doby. Jejich přínosem bývá centralizované ukládání dat, automatizace vybraných operací, vyšší míra kontroly správnosti vstupů a možnost propojení s dalšími personálními nebo mzdovými procesy. Takové systémy lépe odpovídají požadavku na přehlednou, ověřitelnou a dostupnou evidenci a mohou významně snížit administrativní zátěž. Na druhou stranu s sebou zpravidla nesou vyšší technickou, organizační a někdy i ekonomickou náročnost. Nevýhodou

může být také menší pružnost v situacích, kdy je třeba systém přizpůsobit specifickým potřebám určité organizace nebo typu práce [6].

Při porovnání jednotlivých přístupů je zřejmé, že se liší zejména mírou automatizace, přehledností, kontrolovatelností a odolností vůči chybám. Ruční a tabulkové formy evidence jsou sice dostupné a snadno zviditelné, avšak při pravidelném používání mohou zvyšovat časovou náročnost i riziko nepřesností. Specializované systémy naopak přinášejí vyšší úroveň standardizace a automatizace, avšak jejich přizpůsobení konkrétním pracovním podmínkám může být omezené. V prostředí, kde se pracovní režim vyznačuje vyšší mírou autonomie, proměnlivostí nebo kombinací více druhů činností, se proto jako vhodné jeví takové řešení, které spojí přehlednost, jednoduché zadávání dat a automatizaci pouze tam, kde skutečně přináší praktický přínos [6, 7].

Z uvedené analýzy vyplývá, že vhodné řešení evidence pracovní doby by mělo být srozumitelné, přehledné a snadno použitelné, současně však dostatečně spolehlivé a pružné. Mělo by omezovat chybovost při práci s údaji, podporovat jejich následné vyhodnocení a zjednodušovat administrativní část celého procesu. Tyto závěry tvoří východisko pro formulaci požadavků na vlastní softwarové řešení v následující kapitole.

1.3 Požadavky na softwarové řešení

Na základě vymezení evidence pracovní doby a analýzy současných přístupů lze formulovat požadavky na navrhované softwarové řešení. V oblasti softwarového inženýrství představují požadavky základ pro návrh systému, protože vymezují, co má systém poskytovat, jaké potřeby má naplnit a jaké kvalitativní vlastnosti má vykazovat [8]. Pro účely této práce je vhodné rozlišit požadavky funkční a kvalitativní.

1.3.1 Funkční požadavky

Funkční požadavky určují, jaké činnosti má systém vykonávat a jaké služby má uživateli poskytovat. Základním funkčním požadavkem navrhovaného řešení je práce se záznamy pracovní doby. Systém by měl umožňovat evidovat údaje vztahující se k jednotlivým pracovním dnům, zejména začátek a konec práce, přestávky, typ pracovního dne, případně další údaje, které jsou pro vyhodnocení evidence významné. Tyto údaje musí být vedeny tak, aby bylo možné získat přehled o průběhu práce a následně je využít pro administrativní zpracování [1, 2].

Důležitým požadavkem je rovněž možnost následné úpravy zadaných údajů. V praxi může docházet ke změnám rozvrhu práce, k doplnění chybějících informací nebo k opravám již

vytvořených záznamů. Navržený systém by proto měl umožňovat editaci záznamů tak, aby nebylo nutné vytvářet celý výkaz znovu. Tato vlastnost zvyšuje použitelnost systému a lépe odpovídá reálnému průběhu administrativního zpracování pracovních údajů.

Dalším funkčním požadavkem je zpracování vstupních dat, zejména import rozvrhu ze souboru CSV exportovaného ze systému STAG. Aplikace by měla být schopna načíst rozvrhové údaje, ověřit jejich základní správnost a převést je na události uložené v databázi. Tyto události pak tvoří základ pro další práci s evidencí pracovní doby, například pro úpravy v kalendáři, výpočet odpracovaného času, doplnění přestávek a vytvoření výsledného PDF přehledu. Tento požadavek je důležitý proto, že bez importu by bylo nutné jednotlivé rozvrhové položky zadávat ručně, což by zvyšovalo časovou náročnost i riziko chyb.

Mezi funkční požadavky patří také generování výstupů. Cílem systému nemá být pouze interní uchování údajů, ale i vytvoření přehledného výstupu, který bude možné využít pro další administrativní nebo kontrolní účely. Výstup by měl mít srozumitelnou a standardizovanou podobu a měl by umožňovat jednoduchou interpretaci zaznamenaných údajů. V kontextu této práce je důležité zejména to, aby aplikace dokázala převést uložená data do podoby, kterou lze dále používat mimo samotné uživatelské rozhraní aplikace.

1.3.2 Kvalitativní požadavky

Vedle funkčních požadavků je nutné vymezit také kvalitativní požadavky. Ty nepopisují konkrétní funkce systému, například import dat nebo vytvoření PDF výstupu, ale určují, jaké vlastnosti má aplikace při používání mít. U softwarového řešení pro evidenci pracovní doby je důležité nejen to, aby aplikace požadované činnosti umožňovala, ale také to, aby je prováděla přehledně, spolehlivě a způsobem, který nebude pro uživatele zbytečně složitý. Model kvality ISO/IEC 25010 pracuje mimo jiné s charakteristikami, jako jsou použitelnost, spolehlivost a udržitelnost. Tyto vlastnosti jsou pro navrhované řešení důležité, protože ovlivňují jeho praktickou využitelnost i možnost dalšího rozvoje [9].

Prvním důležitým kvalitativním požadavkem je použitelnost systému, která v této práci úzce souvisí s přehledností uživatelského rozhraní. Uživatel by měl být schopen rychle se orientovat v jednotlivých záznamech, porozumět jejich struktuře a provádět běžné operace bez zbytečné náročnosti. V případě evidence pracovní doby je tento požadavek zvlášť důležitý, protože uživatel často pracuje s podobnými údaji, například se začátkem a koncem pracovní doby, přestávkami, typy událostí nebo měsíčními součty. Pokud by bylo rozhraní nepřehledné, mohlo by docházet k chybám při zadávání, kontrole i následné opravě záznamů. Přehledné rozhraní proto pomáhá nejen pohodlnější práci s aplikací, ale také správnosti výsledné evidence.

Druhým významným kvalitativním požadavkem je spolehlivost systému. V této práci se tím rozumí především to, že aplikace musí správně pracovat s uloženými záznamy pracovní doby a musí z nich vytvářet správné výsledky. Pokud uživatel importuje stejný rozvrh nebo provede stejnou úpravu dat, aplikace by měla dojít ke stejnému výsledku. Zároveň musí bezpečně reagovat na chybné nebo neúplné vstupy, například na neplatný CSV soubor, chybně zadaný čas nebo neúplný záznam, aby nedošlo k vytvoření nekonzistentních dat.

Spolehlivost je důležitá proto, že výstupy aplikace slouží jako podklad pro kontrolu pracovní doby. Chyba při uložení události, výpočtu odpracovaných hodin, určení přestávky nebo vytvoření PDF výstupu by se mohla projevit v celém výsledném přehledu. Navrhované řešení proto musí uchovávat data konzistentně, správně provádět výpočty a umožnit uživateli ověřit výsledný stav evidence.

Třetím důležitým kvalitativním požadavkem je udržitelnost. Tento požadavek se týká především vnitřní struktury aplikace a jejího dalšího vývoje. Navrhované řešení by nemělo být technicky zbytečně složité, ale mělo by být rozděleno do přehledných částí, které mají jasně vymezenou odpovědnost. Takové členění usnadňuje orientaci ve zdrojovém kódu a zároveň snižuje riziko, že změna v jedné části aplikace negativně ovlivní jiné části systému.

Udržitelnost je důležitá zejména v situaci, kdy je nutné aplikaci dále upravovat, opravovat nebo rozšiřovat. Pokud je systém navržen přehledně, lze snáze doplnit nové typy událostí, upravit pravidla výpočtu pracovní doby nebo rozšířit výstupní dokumenty. Udržitelný návrh tak podporuje další rozvoj aplikace bez nutnosti zásadně přepracovat celé řešení.

Na použitelnost, spolehlivost a udržitelnost navazuje také požadavek na omezení chybovosti. V běžné praxi vznikají chyby zejména při ručním zadávání, přepisování údajů nebo následném vyhodnocování pracovní doby. Navrhované řešení by proto mělo tato rizika snižovat například přehlednou strukturou dat, kontrolou vstupních hodnot a automatizací opakujících se kroků. Smyslem aplikace není nahradit veškeré rozhodování uživatele, ale omezit ruční a opakující se úkony, které jsou náchylné k chybám. Díky tomu může být práce s evidencí pracovní doby jednodušší, přehlednější a spolehlivější.

1.3.3 Shrnutí požadavků

Z uvedených požadavků vyplývá, že navrhované softwarové řešení má podporovat hlavní činnosti spojené s evidencí pracovní doby a současně musí být použitelné v běžné praxi. Aplikace by měla umožňovat vytváření a úpravu záznamů, import rozvrhových dat, jejich další

zpracování a generování přehledného výstupu. Zároveň by měla být přehledná, spolehlivá, snadno použitelná a navržena tak, aby omezovala vznik chyb při běžné práci.

Takto stanovené požadavky určují, jaké vlastnosti a funkce má navrhovaná aplikace mít. Na jejich základě je možné dále zvolit vhodnou architekturu, rozdělení aplikace do jednotlivých částí a návrhové principy, které budou použity při implementaci. Následující kapitola se proto zaměřuje na softwarovou architekturu a návrhové vzory vhodné pro realizaci navrženého řešení.

1.4 Softwarová architektura a návrhové vzory

Na základě požadavků formulovaných v předchozí kapitole je třeba stanovit takový architektonický přístup, který umožní navrhnout přehledné, udržitelné a spolehlivé softwarové řešení. Volba vhodné softwarové architektury ovlivňuje nejen vnitřní strukturu aplikace, ale také způsob zpracování dat, oddělení odpovědností mezi jednotlivými částmi systému a možnosti jeho dalšího rozvoje. V případě aplikace zaměřené na evidenci pracovní doby je důležité zejména to, aby architektura podporovala přehlednou práci s daty, snadné úpravy jednotlivých částí systému a jednoznačné oddělení uživatelského rozhraní od aplikační logiky.

1.4.1 Softwarová architektura

Softwarovou architekturu lze chápat jako soubor základních rozhodnutí o struktuře systému, jeho hlavních komponentách a vztazích mezi nimi. Určuje, z jakých částí se aplikace skládá, jaké odpovědnosti jednotlivé části nesou a jakým způsobem spolu komunikují. Správně navržená architektura přispívá ke srozumitelnosti systému, usnadňuje jeho údržbu a umožňuje efektivnější rozšiřování funkcionality v budoucnu [10, 11].

V kontextu této práce má volba architektury význam především proto, že navržené řešení pracuje s více typy činností, například se zpracováním vstupních dat, jejich úpravou, vyhodnocením a tvorbou výstupů. Pokud by tyto části nebyly dostatečně odděleny, mohlo by docházet ke zhoršení přehlednosti kódu, vyšší provázanosti jednotlivých komponent a obtížnější testovatelnosti aplikace. Z tohoto důvodu je vhodné zvolit takový architektonický přístup, který umožní rozdělit systém na logicky oddělené části a zároveň zachovat jednoduchost celého řešení.

1.4.2 Návrhový vzor MVVM

Jedním z vhodných návrhových vzorů pro aplikace s grafickým uživatelským rozhraním je MVVM, tedy *Model–View–ViewModel*. Tento přístup rozděluje aplikaci do tří základních částí:

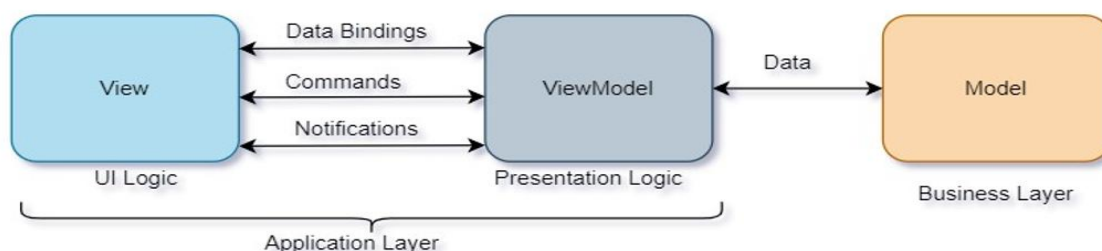
View, *ViewModel* a *Model*. *View* představuje uživatelské rozhraní, tedy obrazovky, dialogová okna a ovládací prvky, se kterými uživatel pracuje. *ViewModel* tvoří prostřední vrstvu mezi uživatelským rozhraním a modelem. Připravuje data pro zobrazení, uchovává stav dané obrazovky a reaguje na uživatelské akce. Model představuje datovou a aplikační část systému, tedy objekty a pravidla, se kterými aplikace pracuje [12, 13].

Vztah mezi těmito částmi je znázorněn na *obrázku 1*. *View* komunikuje s *ViewModelem* především pomocí datových vazeb, příkazů a oznámení. Díky tomu nemusí uživatelské rozhraní přímo obsahovat logiku zpracování dat. *ViewModel* zprostředkovává komunikaci mezi rozhraním a modelem a převádí data do podoby vhodné pro zobrazení uživateli.

V kontextu této práce *Model* nepředstavuje pouze jednu třídu ani přímé napojení na databázi. Jde spíše o část aplikace, která zahrnuje datové objekty, pravidla pro práci s nimi a návaznost na služby zajišťující ukládání, načítání a zpracování údajů. Samotná práce s databází, import rozvrhu nebo výpočty pracovní doby jsou v aplikaci řešeny v samostatných službách a datové vrstvě. *ViewModel* tyto části využívá a jejich výsledky poskytuje uživatelskému rozhraní.

Význam MVVM spočívá především v oddělení uživatelského rozhraní od logiky aplikace. *View* se stará o vzhled a rozložení prvků, zatímco *ViewModel* určuje, jaká data se mají zobrazit a jak má aplikace reagovat na uživatelské vstupy. Tento přístup podporuje přehlednější organizaci kódu, protože jednotlivé části aplikace mají jasněji vymezenou odpovědnost. Zároveň usnadňuje úpravy uživatelského rozhraní, protože změna vzhledu nemusí znamenat zásah do zpracování dat.

Pro desktopové aplikace využívající deklarativní způsob tvorby rozhraní je MVVM přirozeným a často používaným přístupem. Umožňuje efektivně pracovat se stavem aplikace, datovými vazbami mezi *View* a *ViewModelem* a reakcemi na uživatelské akce. Z hlediska požadavků formulovaných v předchozí kapitole je tento vzor vhodný zejména proto, že podporuje přehlednost, udržitelnost a jednoznačné rozdělení odpovědností [12].



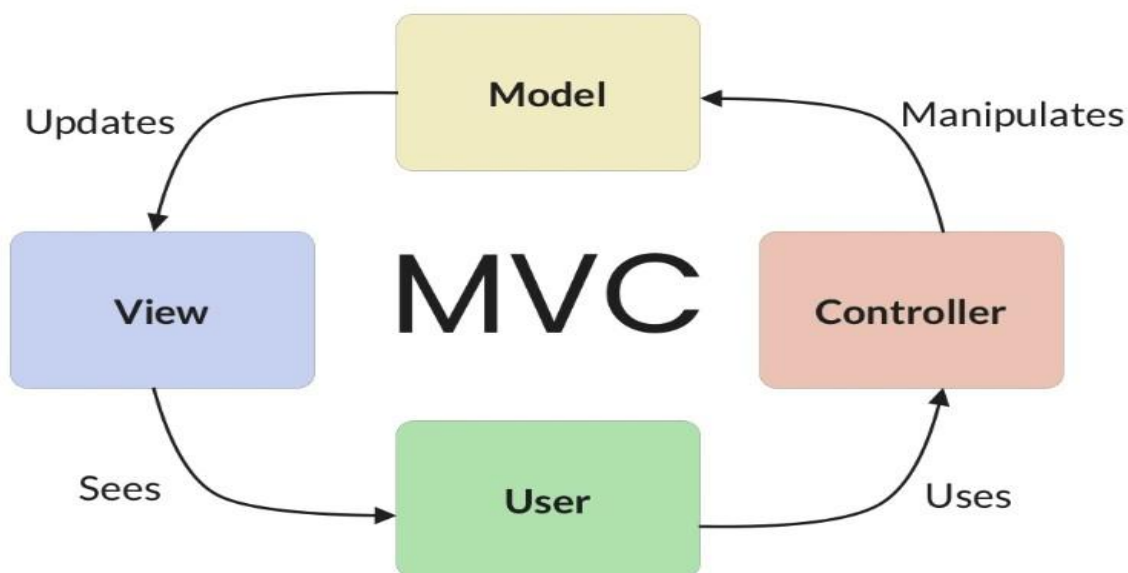
Obrázek 1 Vztah mezi částmi *View*, *ViewModel* a *Model* v architektuře MVVM

1.4.3 Srovnání MVVM a MVC

Vedle vzoru MVVM se v softwarovém inženýrství často používá také návrhový vzor MVC, tedy *Model–View–Controller*. Stejně jako MVVM i tento přístup usiluje o oddělení jednotlivých odpovědností v aplikaci. *Model* představuje data a logiku, *View* slouží k jejich zobrazení a *Controller* zajišťuje zpracování vstupů od uživatele a řízení toku aplikace [14].

MVC je tradičně spojován zejména s aplikacemi, ve kterých *Controller* přímo reaguje na akce uživatele a rozhoduje o tom, jaká data budou zobrazena. Tento přístup je vhodný především v prostředích, kde je potřeba explicitně řídit komunikaci mezi uživatelským vstupem a výstupní vrstvou. V případě moderních desktopových aplikací s deklarativním uživatelským rozhraním však bývá vhodnější MVVM, protože lépe odpovídá principu datových vazeb a oddělení prezentační logiky od samotného zobrazení.

Zatímco MVC kladе důraz na řídicí roli *Controlleru*, MVVM více využívá zprostředkující vrstvu *ViewModelu*, která připravuje data pro prezentaci a omezuje přímou závislost mezi rozhraním a logikou aplikace. Pro aplikaci zaměřenou na evidenci pracovní doby je tento přístup vhodnější zejména proto, že podporuje přehlednější práci se stavem formulářů, záznamů a výstupních údajů. *ViewModel* tak umožňuje soustředit prezentační logiku do samostatné vrstvy a zachovat samotné zobrazení jednodušší a lépe upravitelné.



Obrázek 2 Architektura MVC

1.4.4 Monolitická, modulární a mikroslužbová architektura

Při návrhu softwarového řešení je kromě volby návrhového vzoru důležité také zvolit celkové uspořádání aplikace. Jednou z možností je monolitická architektura, při níž je systém vytvářen

jako jeden celek. Výhodou takového přístupu je jednoduchost vývoje, nasazení i správy, protože aplikace tvoří jedinou spustitelnou jednotku. Nevýhodou však může být menší přehlednost při růstu systému a obtížnější oddělení jednotlivých částí aplikace [15].

Určitou variantu tohoto přístupu představuje modulární monolit. I v tomto případě jde o jednu aplikaci, její vnitřní struktura je však rozdělena do samostatných modulů s jasně vymezenými odpovědnostmi. Takové řešení umožňuje zachovat jednoduchost nasazení a současně zlepšit organizaci systému, jeho přehlednost a udržitelnost. Jednotlivé části aplikace mohou být navrženy tak, aby každá řešila konkrétní oblast funkcionality, aniž by docházelo k jejich zbytečnému provázání [16].

Další možností je mikroslužbová architektura, která rozděluje systém na více samostatně provozovaných služeb. Tento přístup může být výhodný u rozsáhlých systémů s vysokými nároky na škálování, samostatné nasazování jednotlivých částí nebo oddělenou správu více týmů. Současně však přináší vyšší technickou složitost, protože vyžaduje řešení komunikace mezi službami, správu distribuovaného prostředí a větší provozní nároky [17].

Pro aplikaci menšího až středního rozsahu, jejímž cílem je evidence pracovní doby a generování přehledných výstupů, se jako vhodnější jeví jednodušší architektonické řešení. V takovém případě je důležité především zachovat přehlednost, snadnou údržbu a nízkou provozní náročnost, nikoli budovat distribuovaný systém s vlastnostmi, které pro daný účel nejsou nezbytné. Vlastní aplikace proto nemusí využívat mikroslužbový přístup, ale měla by být navržena tak, aby její části byly uvnitř projektu přehledně oddělené.

1.4.5 Zdůvodnění zvoleného řešení

S ohledem na požadavky uvedené v kapitole 1.3 je pro navrhované řešení vhodné zvolit architekturu, která podporuje jednoduchost, přehlednost a oddělení jednotlivých částí systému. Aplikace určená pro evidenci pracovní doby nevyžaduje složitou distribuovanou infrastrukturu ani samostatně provozované služby. Důležitější je, aby byla snadno použitelná, přehledná a aby její jednotlivé části bylo možné dále upravovat nebo rozšiřovat.

Z tohoto důvodu je vhodným řešením modulární monolit. Tento přístup umožňuje zachovat aplikaci jako jeden celek, ale zároveň ji uvnitř rozdělit do logických částí s jasně vymezenou odpovědností. Takové členění je vhodné zejména pro menší a středně rozsáhlé desktopové aplikace, u nichž je důležitá jednoduchost nasazení, nízká provozní náročnost a dobrá udržitelnost. Modulární uspořádání současně omezuje nežádoucí provázanost jednotlivých částí systému.

Pro prezentační část je vhodné využít návrhový vzor MVVM. Tento vzor podporuje oddělení uživatelského rozhraní od části aplikace, která připravuje data pro zobrazení a zpracovává reakce na uživatelské akce. *View* se zaměřuje na vizuální podobu rozhraní, zatímco *ViewModel* zprostředkovává data a stav potřebný pro jeho fungování. Díky tomu lze uživatelské rozhraní měnit nebo rozšiřovat bez nutnosti zásadně zasahovat do ostatních částí systému.

Výhodou tohoto přístupu je také lepší přehlednost zdrojového kódu a snazší údržba aplikace. Oddělení odpovědností umožňuje samostatněji řešit práci s daty, aplikační pravidla a prezentační vrstvu. To je důležité zejména u aplikace, která pracuje s opakovaně upravovanými záznamy, provádí jejich vyhodnocování a vytváří výstupy pro další použití.

Kombinace modulárního monolitu a návrhového vzoru MVVM proto odpovídá charakteru navrhovaného řešení. Umožňuje zachovat jednoduchost aplikace a současně podporuje požadavky na přehlednost, použitelnost, spolehlivost a udržitelnost. Na základě této architektonické volby lze v další kapitole přistoupit k představení technologií vhodných pro implementaci aplikace.

1.5 Použité technologie

Na základě zvolené architektury a návrhových principů je v dalším kroku vhodné vymezit technologie, které byly pro realizaci navrženého řešení použity. Volba konkrétních technologií má přímý vliv na způsob implementace aplikace, její přenositelnost, udržitelnost i možnosti dalšího rozvoje. V případě aplikace zaměřené na evidenci pracovní doby je důležité zvolit takové technologické prostředky, které umožní vytvořit přehledné uživatelské rozhraní, spolehlivě zpracovávat data a generovat odpovídající výstupy.

S ohledem na požadavky formulované v předchozích kapitolách byly pro realizaci aplikace zvoleny technologie platformy .NET, programovací jazyk C#, databázový systém SQLite a framework Avalonia UI. Tyto prostředky společně vytvářejí vhodný základ pro tvorbu desktopové aplikace, která je přehledná, snadno rozšiřitelná a současně technicky nenáročná na nasazení.

1.5.1 Platforma .NET a jazyk C#

Platforma .NET představuje moderní vývojové prostředí určené pro tvorbu různých typů aplikací, včetně desktopových, webových i mobilních řešení. Její výhodou je především široká podpora nástrojů, stabilní ekosystém a možnost vytvářet přenositelná řešení pro více operačních systémů. Pro tuto práci je podstatné, že platforma .NET poskytuje prostředí pro vývoj aplikace kombinující práci s daty, grafické uživatelské rozhraní a generování výstupních dokumentů [2].

Programovací jazyk C# byl zvolen především pro svou čitelnost, silnou typovou kontrolu a dobrou podporu objektově orientovaného přístupu. Tyto vlastnosti přispívají k přehlednější struktuře kódu, usnadňují jeho údržbu a snižují riziko některých typů chyb při vývoji. Výhodou C# je rovněž dobrá integrace s ostatními částmi platformy .NET, což umožňuje efektivně propojit jednotlivé vrstvy aplikace do jednoho funkčního celku [1, 2].

Pro navržené řešení je tato technologie vhodná také proto, že podporuje tvorbu aplikací založených na architektonickém vzoru MVVM. Díky tomu je možné lépe oddělit prezentační vrstvu od aplikační logiky, což odpovídá požadavkům na přehlednost, udržitelnost a testovatelnost systému.

1.5.2 Databázová vrstva: SQLite a jazyk SQL

Součástí navrženého řešení je také práce s daty, která je potřeba ukládat, načítat a dále zpracovávat. Pro tento účel byla zvolena databáze SQLite, tedy lehký relační databázový systém vhodný zejména pro lokální ukládání dat v rámci jedné aplikace. Jeho výhodou je jednoduché nasazení, protože nevyžaduje samostatný databázový server ani složitou správu databázového prostředí [20].

SQLite je vhodnou volbou pro aplikace menšího a středního rozsahu, u nichž je kladen důraz na lokální ukládání dat, jednoduché nasazení a nízké provozní nároky. V kontextu této práce představuje vhodné řešení pro ukládání údajů souvisejících s evidencí pracovní doby, protože umožňuje uchovávat strukturovaná data v přehledné podobě a současně poskytuje dostatečné možnosti pro jejich následné vyhledávání a zpracování.

Databázová vrstva navrhované aplikace není určena pro správu více zaměstnanců ani pro registrační systém. Aplikace je koncipována jako lokální desktopové řešení, ve kterém pracuje jeden uživatel s vlastní evidencí pracovní doby. Databáze proto slouží především k ukládání záznamů pracovních dnů, souvisejících údajů a nastavení potřebných pro správné fungování aplikace. Tento přístup odpovídá charakteru práce, protože cílem není vytvořit personální informační systém pro více uživatelů, ale praktický nástroj pro lokální zpracování evidence.

Při práci s databází je využíván jazyk SQL, který slouží k definici datových struktur i k manipulaci s uloženými daty. SQL umožňuje vytvářet tabulky, upravovat obsah databáze a získávat data podle zadaných podmínek. Pro potřeby této práce je důležitý zejména proto, že poskytuje standardizovaný a srozumitelný způsob práce s relačními daty [21].

1.5.3 Framework Avalonia UI

Pro tvorbu grafického uživatelského rozhraní byl zvolen aplikační framework Avalonia UI. Jedná se o moderní framework pro vývoj desktopových aplikací na platformě .NET, který umožňuje vytvářet multiplatformní uživatelská rozhraní s využitím deklarativního přístupu. To znamená, že vzhled aplikace je možné definovat odděleně od její logiky, což podporuje přehlednost a lepší organizaci celého projektu [12].

Významnou vlastností frameworku Avalonia UI je jeho přirozená podpora architektonického vzoru MVVM. Díky tomu je možné navrhnout uživatelské rozhraní tak, aby bylo odděleno od logiky zpracování dat a jednotlivé vrstvy aplikace nebyly zbytečně provázané. Tento přístup je pro navržené řešení vhodný zejména proto, že aplikace pracuje s větším množstvím údajů, formulářových prvků a výstupních přehledů, u nichž je důležitá přehledná správa stavu a jednoznačné rozdělení odpovědností.

Další výhodou Avalonia UI je možnost vytvářet jednotné uživatelské prostředí napříč více operačními systémy. To odpovídá požadavku na přenositelnost aplikace a současně umožňuje zachovat jednotný způsob ovládání a zobrazení dat bez ohledu na konkrétní platformu. V případě lokální desktopové aplikace je tato vlastnost vhodná také z hlediska případného budoucího rozšíření nebo přenesení řešení na jiné pracovní prostředí.

1.5.4 Podpůrné knihovny

Kromě hlavních technologií byly v projektu využity také podpůrné knihovny, které rozšiřují základní funkčnost aplikace a usnadňují implementaci vybraných úloh. Tyto knihovny nepředstavují samostatné jádro řešení, ale doplňují hlavní technologický celek o prostředky pro efektivnější práci s uživatelským rozhraním, aplikační logikou a tvorbou výstupů.

V souvislosti s architekturou MVVM byla využita knihovna ReactiveUI, která podporuje práci se stavem aplikace, vazbami mezi daty a uživatelským rozhraním a organizací prezentační logiky. Její využití je vhodné zejména v situacích, kdy aplikace pracuje s více formulářovými prvky, změnami hodnot a reakcemi na uživatelské akce. Díky tomu lze část logiky související s prezentací dat soustředit do *ViewModelů* a zachovat přehlednější strukturu projektu [22].

Pro řešení vybraných interakcí v uživatelském rozhraní lze využít také mechanismus Behaviors, který umožňuje oddělit chování rozhraní od samotné vizuální definice prvků. Tento přístup je užitečný například v případech, kdy je potřeba doplnit určité chování ovládacích prvků bez toho, aby byla narušena přehlednost hlavní struktury uživatelského rozhraní [23].

Pro generování výstupních dokumentů byla využita knihovna QuestPDF, která umožňuje vytvářet dokumenty ve formátu PDF v prostředí C# a .NET. Knihovna umožňuje programově definovat strukturu dokumentu, doplnit tabulky a vytvořit výsledný soubor ve formátu PDF. V kontextu aplikace pro evidenci pracovní doby je tato funkcionality důležitá zejména proto, že výsledná data nemají zůstat pouze v databázi, ale mají být dostupná také v podobě samostatného výstupního dokumentu [24].

V případě následné práce s PDF soubory nebo jejich dalším zpracováním lze využít nástroje Ghostscript a Ghostscript.NET, které rozšiřují možnosti práce s tímto formátem v prostředí .NET. Tyto nástroje mohou být využitelné zejména v situacích, kdy je potřeba s vygenerovanými dokumenty dále pracovat, ověřovat jejich podobu nebo je připravovat pro další použití [25, 26].

Využití podpůrných knihoven umožňuje soustředit se na vlastní logiku aplikace a současně snižuje potřebu implementovat běžné technické mechanismy od začátku. Při jejich výběru je důležité zohlednit kompatibilitu se zvolenou architekturou, dlouhodobou udržitelnost a vhodnost pro daný typ aplikace.

1.5.5 Shrnutí použitých technologií

Z výše uvedeného přehledu vyplývá, že zvolené technologie vytvářejí vhodný základ pro realizaci navrženého softwarového řešení. Platforma .NET spolu s programovacím jazykem C# umožňuje vývoj přehledné a udržitelné aplikace. SQLite poskytuje jednoduché prostředí pro lokální správu dat a Avalonia UI podporuje tvorbu uživatelského rozhraní odpovídajícího zvolnému architektonickému přístupu.

Podpůrné knihovny doplňují základní technologické prostředky o další funkce potřebné pro implementaci aplikace. ReactiveUI podporuje práci s prezentační logikou, QuestPDF umožňuje generování výstupních dokumentů a další nástroje rozšiřují možnosti práce s PDF soubory. Celek tak odpovídá požadavkům na aplikaci, která má být lokální, přehledná, technicky nenáročná a dostatečně flexibilní pro zpracování evidence pracovní doby.

Na tuto kapitolu přirozeně navazuje představení vývojového prostředí a dalších nástrojů, které byly při implementaci využity.

1.6 Vývojové prostředí a podpůrné nástroje

Vedle volby vhodné architektury a technologií je pro realizaci softwarového řešení důležité také vývojové prostředí a podpůrné nástroje, které usnadňují implementaci, ladění a správu projektu. Tyto prostředky sice netvoří samotný základ navržené aplikace, ale mají významný vliv na

efektivitu vývoje, přehlednost práce se zdrojovým kódem a možnost ověřování správnosti jednotlivých částí systému.

V rámci této práce byly využity nástroje, které podporují vývoj aplikace na platformě .NET, práci s databází SQLite a průběžnou kontrolu vytvářeného řešení. Jejich výběr vychází z požadavku na přehledné, stabilní a opakovatelné vývojové prostředí, které umožňuje efektivně realizovat jednotlivé části navržené aplikace.

1.6.1 Microsoft Visual Studio

Pro vývoj aplikace bylo využito integrované vývojové prostředí Microsoft Visual Studio. Jedná se o nástroj určený pro tvorbu aplikací na platformě .NET, který poskytuje prostředky pro psaní zdrojového kódu, správu projektové struktury, kompilaci, ladění i testování aplikace. Výhodou tohoto prostředí je zejména jeho komplexnost, díky níž lze většinu vývojových činností provádět v jednom sjednoceném rozhraní [27].

Microsoft Visual Studio podporuje práci s jazykem C# a poskytuje funkce, které usnadňují orientaci ve zdrojovém kódu, odhalování chyb a průběžné ověřování správnosti implementace. Důležitou vlastností je také podpora správy externích balíčků a knihoven, které jsou v projektu využívány. Tím je zajištěna lepší přehlednost závislostí a snadnější správa použitých komponent.

V kontextu této práce je Visual Studio vhodným nástrojem také proto, že podporuje vývoj desktopových aplikací na platformě .NET a umožňuje efektivní práci s více částmi projektu, například s logikou aplikace, datovou vrstvou i uživatelským rozhraním. Jeho využití tak přispívá k přehlednosti vývoje a k rychlejšímu ověřování navrženého řešení.

1.6.2 SQLiteStudio

Pro práci s databází byl využit nástroj SQLiteStudio, který slouží ke správě a kontrole databázových souborů systému SQLite. Tento nástroj umožňuje přehledné zobrazení databázové struktury, práci s tabulkami, spouštění SQL dotazů a ověřování uložených dat mimo samotnou aplikaci [28].

Význam SQLiteStudio spočívá především v tom, že usnadňuje kontrolu databázového schématu a obsahu databáze během vývoje. Díky tomu je možné jednoduše ověřovat správnost navržených tabulek, kontrolovat uložené záznamy a testovat databázové operace nezávisle na uživatelském rozhraní aplikace. Tento postup přispívá k lepší přehlednosti vývoje a usnadňuje identifikaci případných chyb při práci s daty.

V rámci této práce představuje SQLiteStudio vhodný podpůrný nástroj zejména proto, že doplňuje hlavní vývojové prostředí o možnost přímé práce s databází. Tím usnadňuje nejen návrh a kontrolu datové vrstvy, ale také průběžné ověřování správného fungování celé aplikace.

1.6.3 Shrnutí vývojového prostředí a podpůrných nástrojů

Zvolené vývojové prostředí a podpůrné nástroje vytvářejí vhodné podmínky pro realizaci navrženého softwarového řešení. Microsoft Visual Studio poskytuje komplexní prostředí pro implementaci, ladění a správu projektu, zatímco SQLiteStudio umožňuje efektivní kontrolu a správu databázové vrstvy. Jejich kombinace přispívá k přehlednosti vývoje, usnadňuje ověřování jednotlivých částí systému a podporuje stabilní realizaci aplikace.

Na teoretickou část práce navazuje praktická část, která se již zaměřuje na samotný návrh, implementaci a ověření vytvořeného softwarového řešení.

2 PRAKTICKÁ ČÁST

Praktická část této práce se zaměřuje na návrh, implementaci a ověření aplikace určené pro evidenci pracovní doby. Navržené řešení vychází z požadavků formulovaných v teoretické části, zejména z požadavků na práci se záznamy pracovní doby, zpracování vstupních dat, vytvoření přehledného výstupu a omezení chybovosti při běžné práci s údaji. Tato část práce popisuje, jak byly tyto požadavky převedeny do konkrétního návrhu aplikace a jak byly realizovány její hlavní části.

V následujících kapitolách je nejprve popsán návrh řešení a vnitřní struktura aplikace. Dále je uvedena organizace zdrojových souborů, návrh databázové vrstvy a způsob práce s uloženými daty. Samostatná část je věnována algoritmům, které zajišťují import rozvrhu, vytváření a úpravu pracovních záznamů, doplňování přestávek a vyrovnávání pracovní doby. Následně je popsáno uživatelské rozhraní aplikace, které umožňuje s těmito funkcemi pracovat. Závěr praktické části se zaměřuje na testování aplikace a ověření správnosti jejího fungování.

2.1 Návrh řešení

Navržené řešení je desktopová aplikace určená pro lokální zpracování evidence pracovní doby. Aplikace pracuje s údaji o pracovních dnech, událostech, přestávkách, nastavení pracovního režimu a výstupních přehledech. Základním principem návrhu je převést vstupní údaje do jednotné vnitřní struktury, nad kterou lze následně provádět výpočty, kontroly a generování výsledného výstupu.

Důležitou částí návrhu je práce se vstupními daty. Aplikace umožňuje načíst rozvrhové údaje ze souboru CSV a převést je na události uložené v databázi. Tyto události pak tvoří základ pro další zpracování evidence pracovní doby. Součástí návrhu je také možnost ruční úpravy záznamů, protože v praxi může docházet ke změnám pracovního režimu, opravám importovaných údajů nebo doplnění dalších událostí.

Na práci s daty navazuje aplikační logika, která zajišťuje vyhodnocení jednotlivých dnů a správné uspořádání záznamů. Významnou roli zde mají algoritmy pro generování automatických pracovních bloků, doplňování obědových přestávek a vyrovnávání pracovní doby. Tyto části aplikace jsou důležité proto, že samotné uložení událostí nestačí. Údaje je nutné dále zpracovat tak, aby odpovídaly pravidlům evidence pracovní doby a bylo možné z nich vytvořit správný měsíční přehled.

Další částí návrhu je vytvoření výstupu ve formátu PDF. Výstupní dokument slouží jako přehled evidence pracovní doby za zvolené období a obsahuje údaje potřebné pro kontrolu odpracovaného času, přestávek, přesčasů a neodpracované doby. Návrh aplikace proto počítá nejen s interním uložením dat, ale také s jejich převodem do podoby, kterou lze použít mimo samotnou aplikaci.

Uživatelské rozhraní tvoří prezentační část aplikace. Jeho úkolem je zpřístupnit hlavní funkce uživateli, zobrazit uložené záznamy v přehledné podobě a umožnit jejich úpravu. Rozhraní tedy navazuje na datovou a aplikační část systému a poskytuje uživateli prostředky pro práci s importovanými i ručně vytvořenými údaji. Při jeho návrhu byl kladen důraz na srozumitelnost a jednoduché ovládání, aby bylo možné s aplikací pracovat bez zbytečně složitých kroků.

Celkový návrh aplikace je proto rozdělen do několika částí: datové vrstvy, aplikační logiky, algoritmického zpracování, generování výstupů a uživatelského rozhraní. Toto členění umožňuje popsat jednotlivé části řešení samostatně a zároveň ukázat, jak na sebe navazují při běžném zpracování evidence pracovní doby.

2.2 Struktura aplikace

Aplikace je navržena jako desktopové softwarové řešení, které pracuje s lokálně uloženými daty a poskytuje uživateli grafické rozhraní pro správu evidence pracovní doby. Z hlediska vnitřní organizace je rozdělena do několika vzájemně provázaných částí, z nichž každá plní specifickou úlohu. Toto členění vychází ze zvoleného architektonického přístupu a umožňuje oddělit zpracování dat, aplikační logiku a prezentační vrstvu.

Základ systému tvoří datová vrstva, která zajišťuje ukládání a načítání údajů potřebných pro chod aplikace. Na ni navazuje aplikační logika, jejímž úkolem je zpracování vstupních dat, provádění kontrol, vyhodnocování záznamů a příprava údajů pro jejich zobrazení. Samostatnou část představuje vrstva uživatelského rozhraní, která zprostředkovává komunikaci mezi uživatelem a aplikací a umožňuje práci s jednotlivými funkcemi systému.

Důležitou součástí aplikace jsou také podpůrné služby, které zajišťují specifické operace, například import dat, validaci, zpracování vybraných pravidel evidence pracovní doby nebo generování výstupních dokumentů. Tím je dosaženo lepšího oddělení odpovědností mezi jednotlivými částmi systému a současně je usnadněna orientace ve zdrojovém kódu i případné budoucí rozšíření aplikace.

Takto navržená struktura odpovídá požadavku na přehlednost a udržitelnost řešení. Současně vytváří vhodný základ pro implementaci funkcí, které jsou pro evidenci pracovní doby klíčové, a umožňuje návazně popsat organizaci zdrojových souborů, databázovou strukturu a jednotlivé části uživatelského rozhraní.

2.3 Adresářová struktura projektu

Souborová struktura projektu byla navržena tak, aby odpovídala rozdělení aplikace do logických částí a současně podporovala přehlednost zdrojového kódu. Organizace projektu vychází ze zvoleného architektonického přístupu a rozlišuje zejména části určené pro uživatelské rozhraní, aplikační logiku, práci s daty a podpůrné mechanismy. Takové uspořádání usnadňuje orientaci v projektu, zjednodušuje údržbu a vytváří vhodné podmínky pro další rozšiřování aplikace.

Jednotlivé složky projektu sdružují soubory podle jejich funkce v systému. Díky tomu je možné oddělit prezentační vrstvu od datové a aplikační vrstvy a současně zachovat přehledné členění zdrojového kódu. Přehled adresářové struktury projektu je uveden v *příloze B*.

2.3.1 Prezentační vrstva

Prezentační vrstva zahrnuje části projektu, které zajišťují zobrazení dat, reakci na uživatelské akce a vizuální podobu aplikace. Patří sem zejména složky **Views**, **ViewModels**, **Controls**, **Behaviors** a **Converters**.

Složka **Views** obsahuje jednotlivá zobrazení uživatelského rozhraní, například hlavní okna, dialogy a další obrazovky aplikace. Složka **ViewModels** zahrnuje prvky prezentační logiky, které zprostředkovávají komunikaci mezi datovou vrstvou a uživatelským rozhraním. *ViewModely* připravují data pro zobrazení, reagují na uživatelské akce a řídí chování jednotlivých obrazovek.

Složka **Controls** obsahuje vlastní uživatelské ovládací prvky, které jsou opakovaně využívány v různých částech aplikace. Jejich účelem je sjednotit vzhled a chování vybraných částí rozhraní a omezit duplicitu v definici uživatelských prvků. Složka **Behaviors** obsahuje prvky určené pro rozšíření chování uživatelského rozhraní bez nutnosti zasahovat přímo do logiky jednotlivých zobrazení. Tyto komponenty slouží zejména k zachycení nebo úpravě vybraných interakcí mezi uživatelem a aplikací. Složka **Converters** obsahuje převodníky používané při vazbě dat mezi aplikační logikou a uživatelským rozhraním. Jejich úkolem je upravovat datové hodnoty do podoby vhodné pro zobrazení nebo další zpracování v rámci rozhraní aplikace.

2.3.2 Datová a aplikační vrstva

Datová a aplikační vrstva je tvořena zejména složkami **Data**, **Models**, **Services**, **Helpers** a **Messages**. Tyto části projektu zajišťují ukládání dat, jejich zpracování a podporu hlavních funkcí aplikace.

Složka **Data** obsahuje prvky související s databázovou vrstvou aplikace, zejména datový kontext a konfiguraci mapování entit na databázové tabulky. Tato část projektu zajišťuje komunikaci s databází SQLite a podporuje jednotný způsob ukládání a načítání dat.

Složka **Models** zahrnuje datové modely reprezentující základní objekty aplikace. Tyto modely slouží k uchování a předávání údajů, se kterými aplikace pracuje v rámci evidence pracovní doby. Složka **Services** obsahuje služby zajišťující hlavní aplikační operace, například zpracování vstupních dat, validaci, import nebo generování výstupů. Tato vrstva soustřeďuje logiku, která není přímo vázána na uživatelské rozhraní.

Složka **Helpers** obsahuje pomocné třídy a utility, které usnadňují implementaci opakujících se operací. Složka **Messages** slouží k definici zpráv a komunikačních objektů používaných při předávání informací mezi jednotlivými částmi aplikace. Jejím účelem je podpořit oddělení odpovědností a zjednodušit interní komunikaci systému.

```
namespace TeacherScheduleApp.Data
{
    Ссылка: 71 | Egor Razboinikov, 6 дн. назад | Автор: 1, изменений: 7
    public class AppDbContext : DbContext
    {
        Ссылка: 4 | Egor Razboinikov, 20 дн. назад | Автор: 1, изменение: 1
        public DbSet<Employee> Employees { get; set; }
        Ссылка: 7 | Egor Razboinikov, 20 дн. назад | Автор: 1, изменение: 1
        public DbSet<SemesterSettings> SemesterSettings { get; set; }
        Ссылка: 1 | Egor Razboinikov, 20 дн. назад | Автор: 1, изменение: 1
        public DbSet<WeekdaySettings> WeekdaySettings { get; set; }
        Ссылка: 12 | Egor Razboinikov, 20 дн. назад | Автор: 1, изменение: 1
        public DbSet<DaySettings> DaySettings { get; set; }
        Ссылка: 2 | Egor Razboinikov, 20 дн. назад | Автор: 1, изменение: 1
        public DbSet<ImportBatch> ImportBatches { get; set; }
        Ссылка: 52 | Egor Razboinikov, 357 дн. назад | Автор: 1, изменение: 1
        public DbSet<Event> Events { get; set; }
        Ссылка: 11 | Egor Razboinikov, 15 дн. назад | Автор: 1, изменение: 1
        public DbSet<BalanceSelfTrim> BalanceSelfTrims { get; set; }
        Ссылка: 10 | Egor Razboinikov, 15 дн. назад | Автор: 1, изменение: 1
        public DbSet<BalanceTransfer> BalanceTransfers { get; set; }

        Ссылка: 0 | Egor Razboinikov, 6 дн. назад | Автор: 1, изменений: 3
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            if (optionsBuilder.IsConfigured)
                return;

            var appDir = AppContext.BaseDirectory;
            var dbPath = Path.Combine(appDir, "teacherapp.db");

            optionsBuilder.UseSqlite($"Data Source={dbPath}");
        }

        Ссылка: 0 | Egor Razboinikov, 7 дн. назад | Автор: 1, изменений: 6
        protected override void OnModelCreating(ModelBuilder modelBuilder) { ... }
    }
}
```

Obrázek 3 Ukázka třídy AppDbContext

2.3.3 Kořenové soubory projektu

Kořenové soubory projektu obsahují základní konfigurační, spouštěcí a sestavovací části aplikace. Patří sem zejména soubory potřebné pro inicializaci aplikace, správu hlavních nastavení projektu a zajištění jeho sestavení. Tyto soubory vytvářejí nezbytný základ pro provoz celé aplikace a doplňují její vnitřní strukturu o technické prvky potřebné pro vývoj a spuštění.

2.4 Struktura databáze

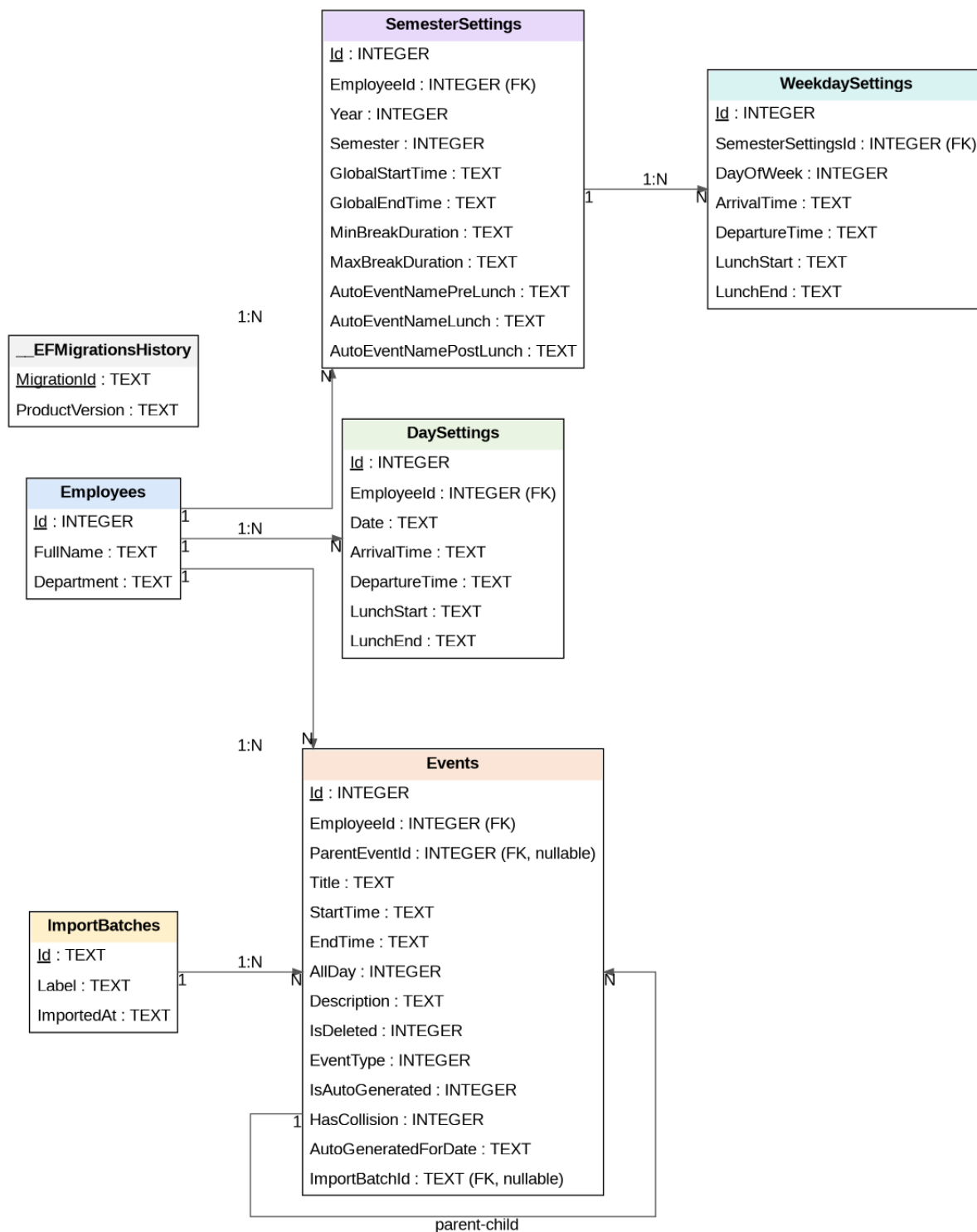
Databázová vrstva aplikace byla navržena jako relační model ukládaný v systému SQLite. Struktura databáze vychází z požadavků na evidenci pracovní doby, správu událostí a uchování nastavení souvisejících s pracovním režimem lokálního uživatele. Návrh databáze byl zaměřen na přehlednost, jednoznačné oddělení jednotlivých typů dat a podporu vazeb mezi hlavními entitami systému.

Aplikace není koncipována jako víceuživatelský personální systém a neobsahuje samostatný registrační mechanismus pro více zaměstnanců. Přesto databázový model obsahuje tabulku **Employees**, která v rámci této aplikace slouží jako technická entita pro uložení základních identifikačních údajů osoby, pro kterou je evidence pracovní doby vytvářena. V běžném použití aplikace pracuje s jedním lokálním záznamem uživatele, nikoli se seznamem registrovaných zaměstnanců.

Na tento lokální uživatelský záznam jsou navázány další části databáze. Tabulka **Events** slouží k ukládání jednotlivých událostí souvisejících s evidencí pracovní doby a obsahuje jak vazbu na lokální uživatelský záznam, tak i volitelnou vazbu na nadřazenou událost a importní dávku. Tabulka **DaySettings** uchovává individuální nastavení pracovního dne pro konkrétní datum a daný uživatelský záznam.

Další část databázového modelu tvoří tabulky **SemesterSettings** a **WeekdaySettings**, které slouží pro ukládání nastavení pracovního režimu v rámci semestru a jednotlivých dnů v týdnu. Tabulka **ImportBatches** eviduje informace o importovaných dávkách dat, na které mohou být jednotlivé události navázány.

Vazby mezi tabulkami jsou realizovány pomocí cizích klíčů, které zajišťují referenční integritu databáze. Tyto vazby však v kontextu aplikace neslouží ke správě více registrovaných uživatelů, ale k jednoznačnému propojení záznamů pracovní doby, nastavení a importovaných dat s lokálním profilem osoby, pro kterou je evidence vedena. Návrh databázové struktury je znázorněn na *obrázku 4*.



Obrázek 4 Relační schéma databáze

2.4.1 Přehled tabulek

Databázový model aplikace je tvořen několika vzájemně propojenými tabulkami, které společně zajišťují ukládání údajů o lokálním uživatelském profilu, událostech, nastaveních pracovního režimu a importovaných datech. Jednotlivé tabulky byly navrženy tak, aby

odpovídaly požadavkům aplikace na přehledné a spolehlivé zpracování evidence pracovní doby.

Podrobné složení jednotlivých tabulek, včetně názvů sloupců a datových typů, je patrné z relačního schématu databáze na *obrázku 4*. Následující text proto jednotlivé tabulky nepopisuje po sloupcích, ale vysvětluje jejich účel v aplikaci a jejich vztah k ostatním částem databázového modelu.

Tabulka **Employees** představuje základní identifikační entitu databázového modelu. V kontextu této aplikace však nepředstavuje seznam registrovaných zaměstnanců ani část personálního systému. Slouží pouze k uchování základních údajů osoby, pro kterou je evidence pracovní doby lokálně vedena. Obsahuje identifikační údaje, zejména jméno a pracoviště, které mohou být následně využity například při generování výstupů. Na tuto tabulku jsou navázány další části databáze, protože záznamy pracovní doby i nastavení pracovního režimu musí být přiřazeny k jednomu konkrétnímu lokálnímu profilu.

Tabulka **Events** slouží k ukládání jednotlivých událostí souvisejících s evidencí pracovní doby. Jedná se o hlavní tabulku pro práci s časovými záznamy v aplikaci. Obsahuje název události, čas začátku a konce, informaci o celodenním charakteru události, popis, typ události a další příznaky související se stavem záznamu. Součástí tabulky jsou také údaje umožňující rozlišit, zda byla událost vytvořena automaticky, zda u ní byla zjištěna kolize, případně zda je navázána na nadřazenou událost nebo na konkrétní importní dávku. Díky této struktuře je možné v jedné tabulce uchovávat různé typy událostí souvisejících s pracovním režimem a současně zachovat jejich přehledné členění.

Tabulka **DaySettings** slouží k ukládání individuálního nastavení pracovního dne pro konkrétní datum a lokální uživatelský záznam. Obsahuje údaje o začátku a konci pracovního dne a dále časové údaje související s přestávkou na oběd. Důležitou součástí této tabulky je příznak **IsManualOverride**, který umožňuje rozlišit, zda bylo nastavení konkrétního dne upraveno ručně. Tato informace je významná zejména pro situace, kdy se denní nastavení odlišuje od výchozích pravidel stanovených pro delší časové období.

Tabulka **SemesterSettings** uchovává obecné nastavení pracovního režimu v rámci konkrétního semestru. Toto nastavení je navázáno na zaměstnance a obsahuje údaje, které určují výchozí parametry pracovního dne, například začátek a konec pracovní doby, minimální a maximální délku přestávky nebo názvy automaticky vytvářených bloků souvisejících s pracovním

režimem. Tabulka tak představuje výchozí konfigurační základ, ze kterého může aplikace při zpracování dat vycházet.

Tabulka **WeekdaySettings** rozšiřuje semestrální nastavení o konkrétní konfiguraci jednotlivých dnů v týdnu. Je navázána na tabulku **SemesterSettings** a umožňuje definovat pracovní režim samostatně pro každý den. Obsahuje údaje o čase příchodu, odchodu a přestávky na oběd pro zvolený den v týdnu. Díky tomu je možné zohlednit rozdíly mezi jednotlivými pracovními dny a přizpůsobit výchozí nastavení skutečnému rozvržení práce.

Tabulka **ImportBatches** eviduje informace o dávkách importovaných dat. Slouží k uchování údajů o tom, kdy byl import proveden a pod jakým označením byl uložen. Tato tabulka umožňuje přiřadit jednotlivé importované události ke konkrétní dávce a tím usnadňuje dohledání původu dat i jejich následné zpracování.

Tabulka **__EFMigrationsHistory** je technická tabulka vytvářená frameworkem Entity Framework Core. Slouží k evidenci provedených migrací databázového schématu a umožňuje sledovat, jaké změny byly v databázi v průběhu vývoje aplikovány. Tato tabulka není součástí vlastní aplikační logiky, ale je důležitá z hlediska správy a údržby databázové struktury.

Z hlediska návrhu databáze lze říct, že jednotlivé tabulky pokrývají základní oblasti potřebné pro fungování aplikace. Datový model zahrnuje lokální uživatelský profil, ukládání jednotlivých událostí, správu nastavení pracovního režimu a evidenci importovaných dat. Díky tomu vytváří vhodný základ pro další zpracování údajů v aplikační logice i pro jejich zobrazení v uživatelském rozhraní.

2.4.2 Shrnutí databázového návrhu

Navržená databázová struktura odpovídá požadavkům aplikace na ukládání údajů o lokálním uživatelském profilu, událostech, nastavení pracovního režimu a importovaných datech. Jednotlivé oblasti dat jsou rozděleny do samostatných tabulek, což přispívá k přehlednosti návrhu, jednodušší správě dat a lepší udržitelnosti celého řešení.

Důležitou vlastností databázového návrhu je také existence vazeb mezi hlavními entitami, které zajišťují referenční integritu a umožňují jednoznačné propojení jednotlivých částí systému. Tyto vazby nejsou určeny pro správu více registrovaných zaměstnanců, ale pro konzistentní propojení pracovních událostí, denních nastavení, semestrálních pravidel a importních dávek s jedním lokálním záznamem osoby, pro kterou je evidence vedena.

Navržená struktura tak vytváří vhodný základ pro implementaci aplikační logiky i pro následnou práci s uživatelským rozhraním aplikace. Současně odpovídá charakteru řešení, které

je určeno jako lokální desktopová aplikace bez registračního systému a bez víceuživatelské správy zaměstnanců.

2.5 Uživatelské rozhraní aplikace

Uživatelské rozhraní aplikace bylo navrženo s důrazem na přehlednost, srozumitelnost a jednoduchost ovládání. Při návrhu rozhraní bylo cílem vytvořit takové prostředí, ve kterém bude možné efektivně pracovat se záznamy pracovní doby, provádět jejich úpravy a současně zachovat dobrou orientaci v datech. Jednotlivé obrazovky aplikace proto vycházejí z jednotných principů ovládání a používají podobné vizuální i funkční prvky.

Rozhraní je členěno tak, aby uživatel mohl snadno přecházet mezi správou událostí, nastavením pracovního režimu a náhledem výsledných výstupů. Součástí návrhu bylo také zajištění návaznosti mezi jednotlivými pohledy, aby aplikace působila jako jeden ucelený systém, a ne jako soubor oddělených oken. Významnou roli hraje také vizuální prezentace dat, která umožňuje zobrazit evidenci pracovní doby v denním, týdenním a měsíčním pohledu.

2.5.1 Hlavní stránka aplikace

Hlavní stránka aplikace představuje základní pracovní prostředí, ve kterém uživatel provádí běžné operace související s evidencí pracovní doby. Rozhraní je rozděleno na dvě hlavní části. Levá část slouží zejména pro ovládání aplikace, výběr data, přístup k hlavním funkcím a zobrazení souhrnných informací o vybraném dni. Pravá část je určena pro kalendářový pohled, ve kterém jsou zobrazovány jednotlivé události a časové bloky pracovní doby.

V levé části hlavní stránky jsou soustředěny nejdůležitější ovládací prvky aplikace, například vytvoření nové události, načtení rozvrhu, otevření globálního nastavení a zobrazení náhledu evidence pracovní doby. Součástí této části je také kalendář pro výběr konkrétního dne, přehled načtených souborů a zobrazení denního nastavení pracovní doby.

Pravá část hlavní stránky slouží k vlastnímu zobrazení evidence pracovní doby v kalendářové podobě. Uživatel zde může sledovat rozložení událostí v čase a přepínat mezi denním, týdenním a měsíčním pohledem podle aktuální potřeby.

Ukázka levé části hlavní stránky je uvedena v *příloze C* a ukázka pravé části hlavní stránky v *příloze D*.

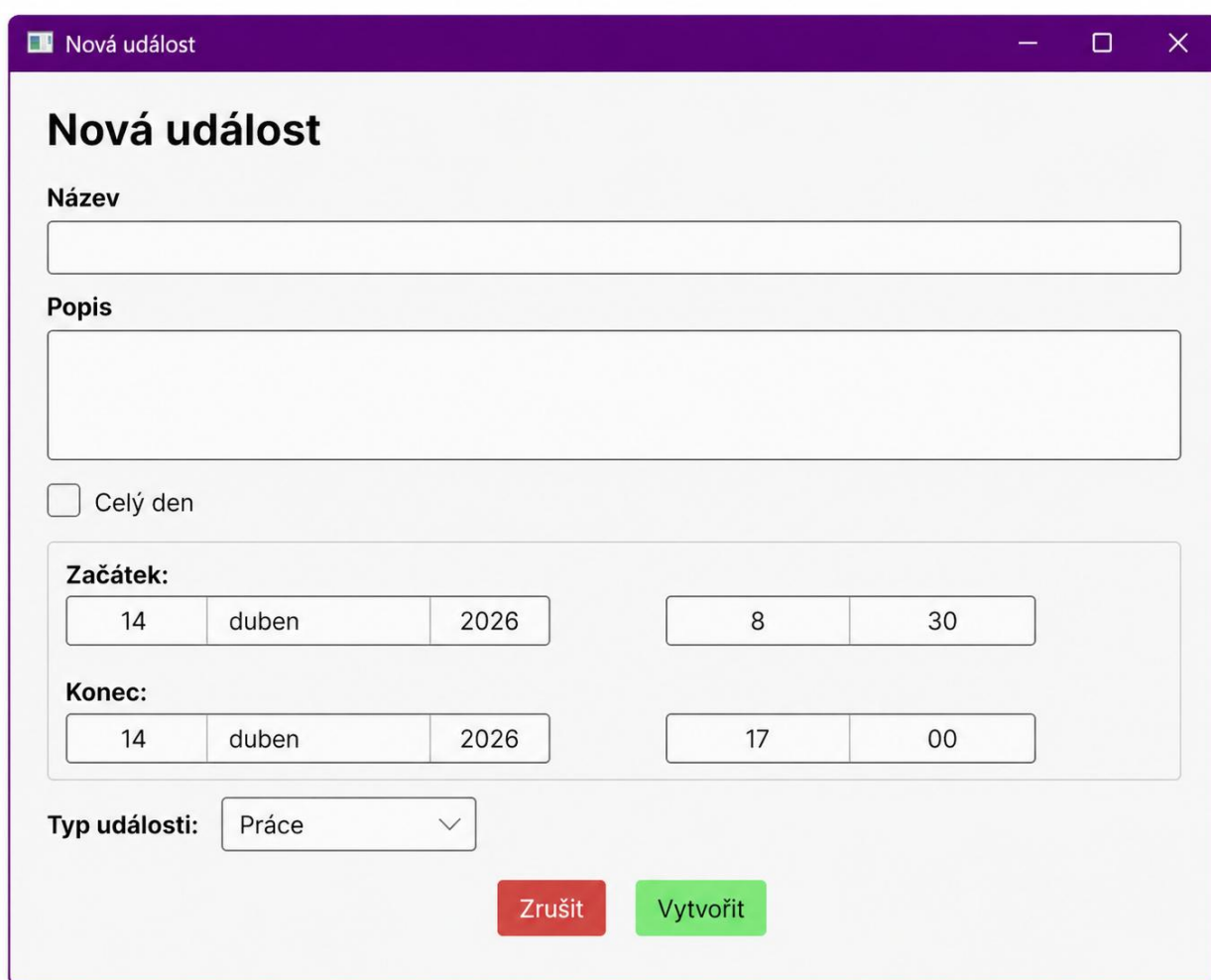
2.5.2 Dialogové okno pro vytvoření události

Dialogové okno pro vytvoření události slouží k zadání nového záznamu do systému. Uživatel zde může vyplnit základní údaje o události, zejména její název, případný popis, časové

vymezení a typ události. Rozhraní dialogu je navrženo tak, aby podporovalo jednoznačné a přehledné zadání všech potřebných údajů.

Součástí dialogu jsou prvky umožňující určit, zda se jedná o celodenní událost, a dále prvky pro zadání data a času jejího trvání. Uživatel také může vybrat typ události, což je důležité pro její další zpracování v rámci evidence pracovní doby. Při ukládání nového záznamu aplikace kontroluje vyplnění povinných údajů, čímž omezuje vznik neúplných nebo neplatných záznamů.

Toto dialogové okno představuje důležitý prvek rozhraní, protože umožňuje rozšiřovat evidenci pracovní doby o nové záznamy přímo z hlavního pracovního prostředí aplikace.



Nová událost

Název

Popis

☐ Celý den

Začátek:

14 duben 2026 8 30

Konec:

14 duben 2026 17 00

Typ události: Práce

Zrušit Vytvořit

Obrázek 5 Dialogové okno pro vytvoření události

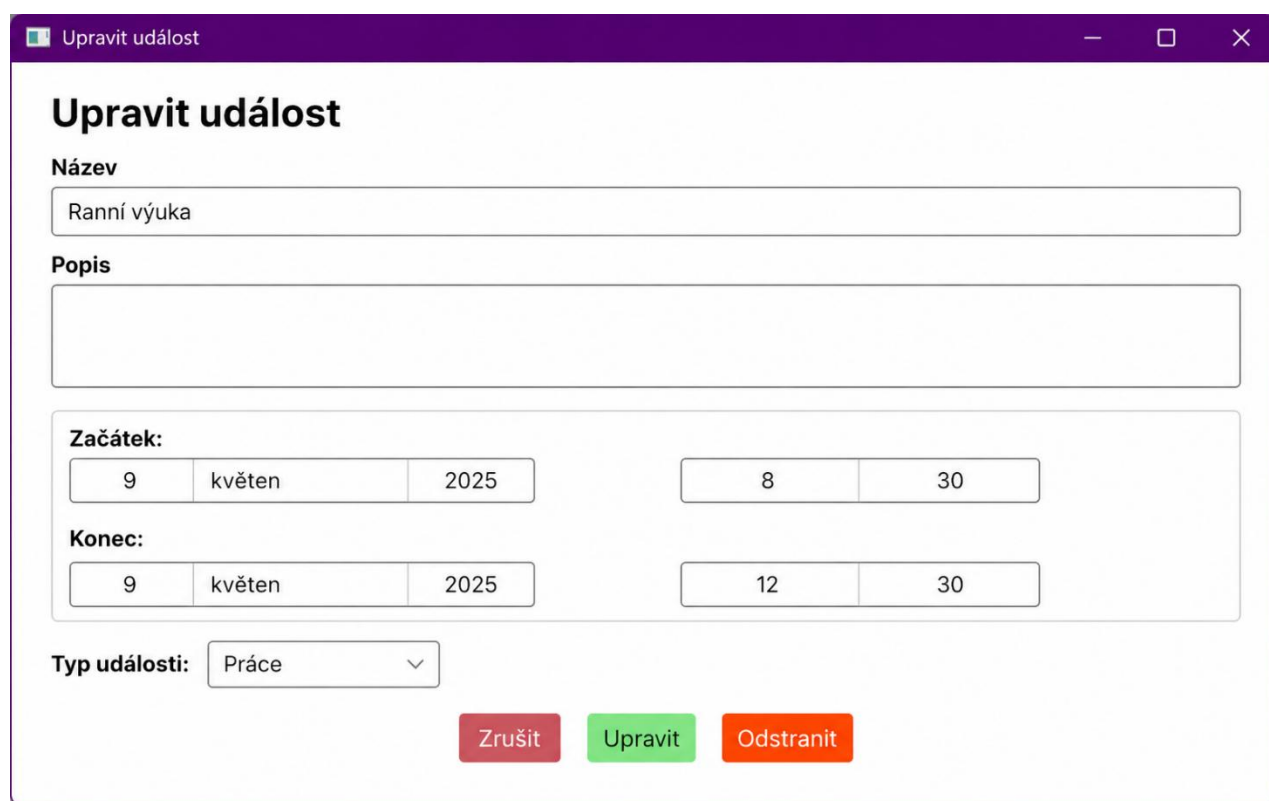
2.5.3 Dialogové okno pro změnu události

Dialogové okno pro změnu události vychází z obdobných principů jako dialog pro vytvoření nového záznamu, avšak je určeno pro úpravu již existujících údajů. Uživatel zde může měnit

název události, její popis, časové vymezení i typ. Součástí dialogu je také možnost odstranění záznamu.

Návrh tohoto okna zajišťuje, aby bylo možné existující události upravovat přehledným a jednotným způsobem. Při změně údajů jsou opět kontrolovány povinné hodnoty a při odstraňování záznamu je vyžadováno potvrzení uživatele, aby se omezilo riziko nechtěného smazání.

Z hlediska uživatelského rozhraní je důležité, že vytváření a úprava událostí využívají podobný princip ovládání. Tím je zachována konzistence aplikace a uživatel nemusí při práci přecházet mezi odlišnými způsoby zadávání údajů.



Obrázek 6 Dialogové okno pro změnu události

2.5.4 Vizualní prezentace pro den

Denní pohled slouží k detailnímu zobrazení konkrétního dne. Umožňuje zobrazit jednotlivé časové úseky v průběhu dne a v této struktuře prezentovat události, které se k danému datu vztahují. Toto zobrazení je vhodné zejména při práci s jednotlivými časově vymezenými záznamy nebo při kontrole konkrétního dne.

Součástí denního pohledu jsou také prvky pro přechod na předchozí a následující den a možnost návratu na aktuální datum. Vizualní členění zobrazení pomáhá odlišit pracovní intervaly od času mimo pracovní režim a umožňuje lepší orientaci v tom, jak jsou události v rámci dne rozloženy.

Denní pohled je významný zejména pro situace, kdy je třeba podrobně zkontrolovat nebo upravit konkrétní denní záznamy. Ukázka denního pohledu aplikace je uvedena v *příloze E*.

2.5.5 Vizualní prezentace pro týden

Týdenní pohled rozšiřuje denní prezentaci na větší časový úsek a umožňuje sledovat události v rámci celého týdne. Toto zobrazení poskytuje přehled o rozložení pracovní doby v několika po sobě jdoucích dnech a usnadňuje orientaci v týdenním kontextu evidence.

Rozhraní týdenního pohledu obsahuje prvky pro přechod mezi jednotlivými týdny a možnost návratu na aktuální týden. Díky společnému zobrazení více dnů je možné snadněji porovnávat pracovní záznamy v rámci týdne a sledovat jejich vzájemné návaznosti. Tento pohled je proto vhodný zejména při posuzování rozložení pracovní doby v širším časovém rámci. Ukázka týdenního pohledu aplikace je uvedena v *příloze F*.

2.5.6 Vizualní prezentace pro měsíc

Měsíční pohled poskytuje nejširší kalendářový přehled a umožňuje zobrazit události v rámci celého měsíce. Zobrazení je založeno na mřížce kalendářních dnů, ve které jsou jednotlivé dny vizuálně odlišeny podle svého charakteru. Víkendy, státní svátky nebo dny náležející do předchozího či následujícího měsíce mohou být zvýrazněny odlišným způsobem, což podporuje rychlejší orientaci uživatele.

Stejně jako v ostatních pohledech je možné přecházet mezi časovými obdobími a vrátit se k aktuálnímu měsíci. Měsíční pohled je vhodný zejména pro celkový přehled o evidovaných událostech a pro orientační kontrolu rozložení pracovní doby v delším období.

Tento způsob prezentace představuje důležitou součást rozhraní, protože umožňuje uživateli sledovat evidenci pracovní doby nejen detailně, ale i v širším časovém kontextu. Ukázka měsíčního pohledu aplikace je uvedena v *příloze G*.

2.5.7 Globální nastavení

Část globálního nastavení slouží ke konfiguraci údajů, které ovlivňují chování aplikace při zpracování evidence pracovní doby a při vytváření výsledných výstupů. Uživatel zde může upravovat výchozí časové parametry pracovního dne, nastavení přestávek a další údaje související s pracovním režimem.

Rozhraní je navrženo tak, aby bylo možné odděleně spravovat nastavení pro různá období, například pro jednotlivé semestry. Vedle časových parametrů obsahuje také údaje, které jsou využívány při tvorbě PDF výstupů nebo při automatickém vytváření vybraných typů událostí.

Tato část aplikace je důležitá zejména proto, že umožňuje přizpůsobit chování systému konkrétním podmínkám bez zásahu do samotné implementace. Ukázka obrazovky globálního nastavení je uvedena v *příloze H*.

2.5.8 Ukázka PDF

Součástí uživatelského rozhraní je také obrazovka určená pro náhled výsledného PDF dokumentu. Tato část umožňuje uživateli zkontrolovat podobu výstupu ještě před jeho uložením. Náhled výstupu podporuje lepší kontrolu správnosti generovaného dokumentu a umožňuje ověřit, zda výsledná podoba odpovídá očekávání.

Uživatel zde může přepínat mezi jednotlivými měsíci a zvolit uložení dokumentu do počítače. Zařazení této funkce přímo do uživatelského rozhraní zvyšuje praktičnost aplikace, protože propojuje práci s evidencí pracovní doby a kontrolu finálního výstupu do jednoho prostředí. Ukázka výsledného PDF dokumentu evidence pracovní doby je uvedena v *příloze I*.

2.5.9 Shrnutí kapitoly

Uživatelské rozhraní aplikace bylo navrženo tak, aby podporovalo přehlednou a efektivní práci s evidencí pracovní doby. Jednotlivé části rozhraní pokrývají hlavní činnosti uživatele, od správy událostí přes nastavení pracovního režimu až po kontrolu výsledných výstupů. Důraz byl kladen na jednotnost ovládání, návaznost jednotlivých obrazovek a srozumitelnou vizuální prezentaci dat.

Navržené rozhraní tak vytváří vhodné prostředí pro každodenní práci s aplikací a současně podporuje funkce, které jsou pro evidenci pracovní doby zásadní. Na tuto kapitolu navazuje část věnovaná algoritmům, které zajišťují zpracování vstupních dat a vyhodnocení evidence pracovní doby.

2.6 Algoritmy

Tato kapitola popisuje hlavní algoritmy, které zajišťují automatické zpracování údajů v aplikaci. Předchozí kapitoly se věnovaly databázové struktuře a uživatelskému rozhraní, zatímco tato část vysvětluje, jak aplikace pracuje s načtenými událostmi, jak doplňuje automatické pracovní bloky, jak kontroluje obědové přestávky a jak připravuje výsledná data pro evidenci pracovní doby.

V aplikaci mají zásadní význam dva procesy. Prvním z nich je algoritmus vyrovnávání hodin. Ten se stará o to, aby odpracovaný čas odpovídal požadovanému pracovnímu fondu a aby se přitom zbytečně neměnily ručně zadané nebo importované události. Algoritmus proto upravuje především automaticky vytvořené pracovní bloky, například jejich začátek nebo konec.

Druhým důležitým procesem je import rozvrhu ze systému STAG. Tento algoritmus načítá vstupní soubor ve formátu CSV, kontroluje jeho obsah, vytváří z něj výukové události a ukládá je do databáze. Po importu aplikace automaticky navazuje dalším zpracováním, zejména regenerací pracovních bloků, kontrolou obědových přestávek a vyrovnaním pracovní doby.

Pro lepší přehlednost nejsou algoritmy znázorněny jedním rozsáhlým diagramem, ale několika menšími blokovými schématy. Každé schéma popisuje pouze jednu část zpracování, aby bylo možné snadněji pochopit návaznost jednotlivých kroků.

2.6.1 Algoritmus vyrovnávání hodin

Algoritmus vyrovnávání hodin slouží k tomu, aby evidence pracovní doby odpovídala požadovanému pracovnímu fondu. Zároveň musí zachovat události, které zadal uživatel ručně nebo které byly převzaty z importovaného rozvrhu. Z tohoto důvodu algoritmus nepracuje se všemi událostmi stejným způsobem. Ručně zadané a importované události bere jako pevný základ a upravuje především automaticky vytvořené pracovní bloky.

Vyrovňávání může být spuštěno pro konkrétní týden, pro celý měsíc nebo pro rozsah dnů, které byly změněny například po importu dat nebo po ruční úpravě události. Nejprve se určí pracovní dny, se kterými má algoritmus pracovat. Do výpočtu se nezahrnují víkendy a státní svátky, protože pro tyto dny se běžný pracovní fond nepočítá.

Základní zpracování probíhá po týdnech. Pro každý pracovní den se vypočítá, kolik času bylo započteno jako práce a zda daný den obsahuje přebytek nebo nedostatek hodin. Aplikace následně porovnává stav celého týdne. Pokud je v týdnu odpracováno více hodin, než odpovídá požadovanému fondu, algoritmus se pokusí přebytek odebrat z automatických pracovních bloků. Pokud naopak v týdnu hodiny chybí, algoritmus se pokusí automatické bloky prodloužit.

Úpravy jsou prováděny v pětiminutových krocích. Díky tomu jsou změny pravidelné a předvídatelné. Algoritmus se nejprve snaží upravovat začátek nebo konec pracovního dne, protože tyto části jsou pro automatické vyrovnání nejvhodnější. Pokud den obsahuje ručně zadané události na začátku i na konci dne, je považován za zamčený. U takového dne nelze jednoduše zkrátit automatický pracovní blok na okraji dne, a proto se přebytek řeší pomocí jiných dnů ve stejném týdnu.

Před samotným vyrovnáváním se nejprve odstraní nebo vrátí dříve provedené pomocné úpravy, pokud již neodpovídají aktuálním datům. To je důležité hlavně při opakovaném spuštění algoritmu. Aplikace tak nezačíná z již částečně upraveného stavu, ale snaží se znovu vyhodnotit aktuální situaci podle skutečně uložených událostí.

Pokud má týden přebytek hodin, algoritmus se pokusí zkrátit automatické pracovní bloky v dnech, kde je to možné. Zkracování probíhá rovnoměrně, aby nebyl zbytečně upraven pouze jeden den. Jestliže přebytek vznikne v zamčeném dni, kde není možné přímo upravit začátek nebo konec dne, aplikace se pokusí najít jiné dny ve stejném týdnu, které lze zkrátit. Tím se přebytek vyrovná v rámci týdne, aniž by se zasahovalo do ručně zadaných událostí.

Pokud má týden nedostatek hodin, algoritmus postupuje opačně. Snaží se doplnit chybějící čas prodloužením automatických pracovních bloků. Podle dostupného prostoru může prodloužit začátek pracovního dne směrem dříve nebo konec pracovního dne směrem později. I v tomto případě se postupuje v pětiminutových krocích a pouze tam, kde to nastavení dne a existující události umožňují.

Samostatnou část algoritmu tvoří kontrola obědové přestávky. Po změně začátku nebo konce pracovního dne je nutné ověřit, zda oběd stále leží uvnitř pracovního okna a zda nekoliduje s jinou událostí. Aplikace určuje počet obědových přestávek podle délky pracovního okna, případně podle rozsahu ručně zadané práce. Pokud délka nepřesáhne čtyři hodiny, oběd se automaticky nevytváří. Při délce přesahující čtyři hodiny se vytváří jedna obědová přestávka a při délce přesahující dvanáct hodin se vytvářejí dvě obědové přestávky.

Po doplnění nebo úpravě oběda aplikace rozdělí automatické pracovní bloky tak, aby se s obědem nepřekrývaly. Pokud je oběd neplatný, například leží mimo pracovní dobu, algoritmus ho odstraní nebo se pokusí najít vhodné místo v rámci pracovního dne. Cílem je, aby výsledné události byly časově správné a aby nevznikaly překryvy mezi prací a přestávkou.

Celý proces vyrovnávání se může několikrát zopakovat. Po každém průchodu se vytvoří kontrolní otisk aktuálního stavu, který zahrnuje události, denní nastavení a pomocné informace o přesunech nebo zkrácení. Pokud se stav po dalším průchodu již nemění, je algoritmus považován za stabilizovaný a vyrovnávání se ukončí.

Po týdenním vyrovnání následuje ještě kontrola v rámci měsíce. Ta řeší zejména drobné odchylky, neúplné týdny na začátku nebo na konci měsíce a správné umístění obědových přestávek. Výsledkem algoritmu je evidence pracovní doby, která zachovává ruční zásahy uživatele, upravuje pouze automaticky vytvořené části a umožňuje opakované spuštění bez vzniku duplicitních nebo nekonzistentních změn. Zjednodušená bloková schémata algoritmu vyrovnávání hodin jsou z důvodu lepší čitelnosti rozdělena do několika částí. Celkový průběh algoritmu je uveden v přílohách *J.1* a *J.2*, týdenní vyrovnání pracovní doby v přílohách *K.1* a *K.2* a kontrola obědových přestávek v přílohách *L.1* až *L.3*.

2.6.2 Algoritmus importu rozvrhu ze STAG

Algoritmus importu rozvrhu ze systému STAG slouží k tomu, aby uživatel nemusel výukové události zadávat ručně. Vstupem algoritmu je soubor CSV exportovaný ze systému STAG. Výstupem jsou události uložené v databázi aplikace, se kterými lze dále pracovat stejně jako s ostatními záznamy v kalendáři.

Import začíná načtením souboru. Protože soubor může být uložen v různém kódování, aplikace se nejprve pokusí rozpoznat vhodné kódování a následně načte jednotlivé řádky. Pokud je soubor prázdný, import se neprovede a aplikace zobrazí chybovou informaci. Poté se zkontroluje hlavička souboru a počet sloupců. Pokud soubor nemá požadovanou strukturu, není možné v importu pokračovat.

Následně aplikace postupně zpracovává jednotlivé řádky CSV souboru. Každý řádek je nejprve rozdělen na sloupce. Při rozdělování se zohledňují uvozovky, aby nedošlo k chybnému rozdělení textu, který může obsahovat oddělovač. Pokud je řádek poškozený nebo obsahuje nedostatečný počet sloupců, je přeskočen a informace o chybě se uloží do importního reportu.

U platně načteného řádku aplikace kontroluje data, čas začátku a konce, rozsah týdnů a další údaje potřebné pro vytvoření události. Pokud některý z těchto údajů není platný, řádek se nepoužije. Stejně tak se odmítne řádek, ve kterém je konec události dříve než začátek nebo je s ním shodný. Tím se zabrání vytvoření časově nesprávných záznamů.

Z každého platného řádku se následně vytvoří jedna nebo více výukových událostí. Algoritmus bere v úvahu období platnosti, den v týdnu, rozsah týdnů a paritu týdne. Událost je vytvořena pouze pro taková data, která splňují všechny podmínky uvedené v rozvrhu. Nově vytvořené události jsou označeny jako výukové události a nejsou považovány za automaticky generované pracovní bloky, protože jejich zdrojem je importovaný rozvrh.

Před uložením nové dávky importu aplikace určí časový rozsah importovaných dat. V tomto rozsahu vyhledá starší importované události a označí je jako smazané. Nejsou tedy okamžitě fyzicky odstraněny z databáze, ale přestanou být používány jako aktivní záznamy. Díky tomu je možné stejný rozvrh importovat opakovaně bez vzniku duplicit.

Poté aplikace vytvoří záznam importní dávky. Tento záznam obsahuje identifikátor dávky, název importovaného souboru a čas importu. Všechny nově vytvořené události jsou s touto dávkou propojeny. To umožňuje později dohledat, z jakého importu konkrétní události pocházejí.

Po uložení výukových událostí import nekončí. Aplikace spustí navazující zpracování změněného rozsahu. V rámci tohoto kroku se regenerují automatické pracovní bloky, zkontrolují se obědové přestávky a spustí se vyrovnání pracovní doby. Tím je zajištěno, že importovaná data jsou ihned převedena do podoby použitelné pro evidenci pracovní doby.

Výsledkem algoritmu je sada výukových událostí uložených v databázi, report o průběhu importu a aktualizovaná evidence pracovní doby pro dotčené období. Výhodou tohoto postupu je, že aplikace dokáže zpracovat běžný import ze STAGu, rozpoznat chybové řádky, nahradit předchozí importovaná data a následně automaticky připravit pracovní evidenci pro další použití. Bloková schémata znázorňující průběh importu rozvrhu ze systému STAG a navazující zpracování importovaných dat jsou uvedena v *přílohách M a N*.

2.7 Testování aplikace

Tato kapitola se zaměřuje na ověření funkčnosti vytvořené aplikace. Cílem testování bylo zjistit, zda aplikace správně podporuje celý pracovní postup, tedy import rozvrhu z externího souboru, následné zpracování a vyrovnání pracovní doby podle pravidel evidence pracovní doby, a nakonec vytvoření výsledného výstupu ve formátu PDF. Současně bylo ověřováno, zda aplikace vykazuje stabilní chování na podporovaných platformách a zda je její uživatelské rozhraní použitelné i při delších nebo výpočetně náročnějších operacích.

Testování probíhalo průběžně v průběhu implementace i po dokončení hlavních funkcí aplikace. V závěrečné fázi byly provedeny souvislé testy nad vybranými scénáři, které pokrývaly běžné i hraniční situace. Pozornost byla věnována především správnosti denních, týdenních a měsíčních součtů, korektnímu chování algoritmu vyrovnávání, správnému zpracování importovaných dat a čitelnosti výsledných výstupů.

2.7.1 Cíl a kritéria přijetí

Hlavním cílem testování bylo ověřit, že aplikace odpovídá požadavkům stanoveným v předchozích kapitolách a že ji lze použít pro praktickou práci s evidencí pracovní doby. Za přijatelný výsledek bylo považováno takové chování systému, při němž aplikace bez chyby načte vstupní CSV soubor, vytvoří odpovídající události, správně provede jejich další zpracování a následně umožní vytvoření přehledného výstupu ve formátu PDF.

Dále bylo požadováno, aby algoritmus správně vyrovnával pracovní dobu v rámci týdne a současně korektně zohledňoval neúplné týdny na začátku a na konci měsíce. Výstupní PDF dokument musel obsahovat správně strukturovanou tabulku evidence pracovní doby, včetně součtů, hlaviček a zvýraznění relevantních dnů, například státních svátků. Současně bylo

ověřováno, že aplikace vykazuje srovnatelné chování na operačních systémech Windows a Linux a že uživatelské rozhraní zůstává dostatečně responzivní i při delším zpracování.

2.7.2 Testovací prostředí

Aplikace byla testována na dvou hlavních platformách, a to na operačním systému Windows 11 a na systému Ubuntu 22.04 LTS. V obou případech byla použita platforma .NET 8, uživatelské rozhraní postavené na frameworku Avalonia UI a lokální databázové úložiště SQLite.

Na platformě Windows probíhalo testování v prostředí Windows 11 verze 23H2. Na platformě Linux bylo testování realizováno v prostředí Ubuntu 22.04 LTS, a to s běžnou systémovou konfigurací desktopového prostředí. V obou případech byla aplikace provozována nad reálnou databází SQLite uloženou v souboru a byly používány testovací vstupy odpovídající běžnému provozu aplikace.

2.7.3 Metodika ověřování funkčnosti

Pro ověření funkčnosti aplikace byla použita kombinace scénářového testování, manuální kontroly a kontroly výsledných dat. Testování nebylo zaměřeno pouze na izolované části aplikace, ale především na celé pracovní postupy, které odpovídají reálnému použití systému. Ověřována byla zejména návaznost mezi importem dat, vytvořením a úpravou událostí, automatickým vyrovnáváním pracovní doby a generováním PDF výstupů.

Scénářové testování bylo zvoleno proto, že aplikace zahrnuje několik navazujících kroků a výsledek jedné operace často ovlivňuje další zpracování. Například import rozvrhu ze souboru CSV nevytváří pouze samostatné události, ale zároveň ovlivňuje denní nastavení, automatické pracovní bloky, obědové přestávky a následné vyrovnání pracovní doby. Z tohoto důvodu bylo důležité ověřovat aplikaci jako celek a sledovat, zda jednotlivé části systému spolupracují správně.

Manuální kontrola byla využita zejména při ověřování uživatelského rozhraní, správného zobrazení údajů a reakce aplikace na uživatelské vstupy. Při této kontrole bylo sledováno, zda jsou ovládací prvky srozumitelné, zda se záznamy zobrazují ve správných pohledech a zda aplikace reaguje očekávaným způsobem při vytváření, úpravě nebo odstranění událostí. Součástí manuální kontroly bylo také ověření náhledu PDF dokumentu a práce s nastavením pracovního režimu.

Při testování byly používány jak validní vstupní soubory, tak záměrně upravené chybové vstupy. Tím bylo možné ověřit nejen běžné scénáře, ale také reakci aplikace na neplatná data, nesprávné formáty času, chybějící sloupce nebo kolizní situace v datech. U výsledných PDF dokumentů

nebyla posuzována binární shoda souborů, ale obsahová správnost, přehlednost a úplnost výstupu.

Testování bylo zaměřeno jak na správnost datového zpracování, tak na použitelnost výsledného řešení v praxi. Ověřovány byly tedy nejen správné součty a výstupy, ale také plynulost práce s aplikací, srozumitelnost chybových hlášení a stabilita systému při opakovaném provádění stejných operací. Tento způsob ověřování umožnil zachytit nejen běžné chyby ve zpracování dat, ale také praktické problémy, které by se mohly projevit při každodenním používání aplikace.

2.7.4 Předpokládané rozdíly mezi platformami

Při testování bylo zohledněno, že mezi platformami Windows a Linux mohou vznikat drobné vizuální rozdíly. Tyto rozdíly se mohou týkat zejména použitých systémových fontů, metrik textu, šířky některých prvků nebo drobných odlišností ve vykreslování uživatelského rozhraní. Podobně se mohou projevit i malé rozdíly ve vzhledu rámečků, posuvníků nebo dialogových prvků.

Za přijatelný stav bylo považováno, pokud tyto rozdíly neovlivní obsahovou správnost aplikace, čitelnost rozhraní ani použitelnost výstupních dokumentů. V praxi to znamená, že text musí být na obou platformách čitelný, ovládací prvky musí být dostupné a nesmí docházet k překrývání obsahu. U exportovaných PDF dokumentů musel zůstat zachován stejný obsah, i když se mohly mírně lišit některé vizuální detaily dané platformou.

2.7.5 Testovací scénáře

Testovací scénáře byly navrženy tak, aby pokrývaly běžné situace při používání aplikace i vybrané hraniční případy. Každý scénář vycházel ze stejného principu: nejprve byla připravena vstupní data nebo odpovídající stav databáze, následně byla provedena konkrétní akce v aplikaci a poté byl porovnán očekávaný a skutečný výsledek.

První skupinu scénářů tvořily testy zaměřené na standardní provoz aplikace. V těchto scénářích byl použit platný export CSV ze systému STAG za jeden kalendářní měsíc. Po dokončení importu bylo ověřováno, zda došlo ke správnému vytvoření událostí, zda byly správně vypočteny denní a týdenní součty a zda aplikace automaticky doplnila přestávku na oběd a upravila případné přesahy pouze v automaticky generovaných blocích. Na tento scénář navazovalo ověření opakovaného importu stejného období, při němž bylo sledováno, zda nedochází ke vzniku duplicit a zda v databázi zůstává pouze aktuální importní dávka.

Další skupinu tvořily negativní scénáře. Zde byly záměrně připraveny neplatné nebo neúplné vstupy, například soubory s chybně zadaným datem, časem nebo chybějícím povinným sloupcem. V těchto případech bylo sledováno, zda aplikace správně reaguje chybovým hlášením nebo bezpečně zpracuje problematické řádky, aniž by došlo k porušení konzistence uložených dat.

Samostatná skupina testů byla zaměřena na situace související s pravidly evidence pracovní doby. Byly ověřovány scénáře zahrnující neúplné týdny, státní svátky, přesahy mezi začátkem a koncem měsíce, kombinaci výuky a služební cesty v jednom dni nebo krátkodobou nepřítomnost zasahující do pracovní doby. V těchto scénářích bylo důležité zejména potvrdit správnost výpočtu odpracovaného a neodpracovaného času a korektní chování algoritmu vyrovnávání.

Dále byly testovány i scénáře zaměřené na ruční zásahy uživatele, například změna okrajů dne nebo délky oběda. V těchto případech bylo sledováno, zda aplikace po úpravě správně regeneruje automatické bloky, zachová platné denní nastavení a nevytváří nekonzistentní záznamy.

Poslední skupina testů byla věnována exportu, výkonu, použitelnosti a ověření dříve problematických stavů. Byla ověřována správnost PDF výstupu, rychlost importu a exportu, chování aplikace při zvětšeném měřítku uživatelského rozhraní, reakce na pokus o ukládání do chráněné složky a také stabilita aplikace při opakovaném importu stejného souboru. Součástí této části bylo rovněž ověření pravidel pro automatické vytváření obědových přestávek, zejména vytvoření jedné přestávky u pracovních dnů delších než 4 hodiny a vytvoření dvou přestávek u pracovních dnů v délce alespoň 12 hodin. Testy zároveň potvrdily, že aplikace zachovává správné chování i při delších operacích a že uživatelské rozhraní zůstává dostatečně responzivní.

2.7.6 Souhrnná tabulka ověřených scénářů a řešených stavů

Souhrnná tabulka zachycuje hlavní ověřené scénáře a řešené chybové nebo hraniční stavy, které byly během testování aplikace kontrolovány. U jednotlivých položek je uvedena oblast ověření, stručný scénář, popis ověřovaného problému a způsob jeho řešení v aplikaci.

Do tabulky byly zařazeny běžné provozní scénáře, například import dat ze systému STAG, opakovaný import stejného období nebo export výsledného PDF dokumentu. Vedle nich jsou uvedeny také scénáře zaměřené na problematické vstupy, kolizní situace, ruční úpravy uživatele, denní strukturu pracovního dne a pravidla pro automatické vytváření přestávek. Tím

tabulka poskytuje přehled nejen o funkčnosti aplikace, ale také o tom, že byly ověřeny situace, které mohly při vývoji vést k chybám nebo nekonzistentnímu chování systému. Souhrnná tabulka ověřených scénářů a řešených stavů je uvedena v *příloze O*.

2.7.7 Shrnutí testování a známá omezení

Provedené testování ukázalo, že aplikace je schopna zvládnout hlavní pracovní scénáře, pro které byla navržena. Ověřen byl import rozvrhu ze systému STAG, zpracování chybových vstupů, opakovaný import stejného období, vytváření a úprava událostí, automatické generování obědových přestávek, ukládání údajů do lokální databáze SQLite a tvorba výsledného PDF výstupu.

Testování zároveň potvrdilo, že aplikace zachovává konzistentní stav dat i v situacích, kdy vstupní CSV soubor obsahuje neplatné nebo neúplné údaje. V těchto případech aplikace problematické řádky nepoužije pro vytvoření událostí nebo import korektně odmítne. Tím se omezuje riziko vzniku nekonzistentních záznamů v databázi.

Výsledné dokumenty ve formátu PDF se mohly v některých případech lišit v technických detailech, například v metadatech nebo drobných typografických vlastnostech, avšak jejich obsahová správnost zůstávala zachována. Z tohoto důvodu bylo při testování PDF dokumentů posuzováno především jejich věcné a vizuální vyznění, nikoli binární shoda souborů.

Za známá omezení lze považovat především závislost některých vizuálních detailů na konkrétní platformě a použitých systémových fontech. Dalším omezením je skutečnost, že některé operace mohou při větším rozsahu dat vyžadovat delší dobu zpracování. Toto omezení však nemění správnost výsledných dat ani základní použitelnost aplikace.

Z výsledků testování lze uzavřít, že aplikace splňuje stanovené požadavky a je použitelná pro praktickou práci s evidencí pracovní doby. Testy současně potvrdily správnost hlavních algoritmů, konzistenci uložených dat a správnost výsledných výstupů.

ZÁVĚR

Cílem této bakalářské práce bylo navrhnout, implementovat a ověřit aplikaci určenou pro evidenci pracovní doby. Navržené řešení bylo vytvořeno s důrazem na přehlednost, srozumitelnost, spolehlivost a omezení chybovosti při zpracování údajů. Výsledkem práce je desktopová aplikace, která umožňuje evidenci a úpravu záznamů pracovní doby, import vstupních dat a generování výsledných výstupů ve formátu PDF.

V teoretické části byly vymezeny základní principy evidence pracovní doby, analyzovány současné přístupy k jejímu vedení a formulovány požadavky na vhodné softwarové řešení. Následně byly popsány architektonické a technologické prostředky použité při realizaci aplikace. Tato část vytvořila teoretický základ pro návrh praktického řešení.

Praktická část práce se zaměřila na návrh, implementaci a ověření vytvořené aplikace. Byla popsána struktura systému, databázový model, uživatelské rozhraní i algoritmy, které zajišťují klíčové operace aplikace. Zvláštní pozornost byla věnována zejména algoritmu vyrovnávání hodin a algoritmu importu rozvrhu ze systému STAG, protože právě tyto části představují základní logiku celého řešení.

Součástí práce bylo rovněž testování aplikace. Testovací scénáře byly zaměřeny na běžné i hraniční situace, včetně importu dat, vyrovnávání pracovní doby, generování PDF výstupů a práce s uživatelským rozhraním. Výsledky testování ukázaly, že aplikace splňuje stanovené požadavky, správně zpracovává data a poskytuje stabilní a přehledné výstupy. Současně bylo ověřeno, že aplikace vykazuje srovnatelné chování na platformách Windows i Linux.

Přínosem práce je vytvoření funkčního softwarového řešení, které může přispět ke zjednodušení evidence pracovní doby a ke zefektivnění její administrativní části. Do budoucna by bylo možné aplikaci dále rozšířit například o další možnosti exportu, rozšířené testování nebo napojení na další informační systémy.

POUŽITÁ LITERATURA

1. ČESKO, 2006. **Zákon č. 262/2006 Sb., zákoník práce**. In: *Zákony pro lidi* [online]. [cit. 2026-04-26]. Dostupné z: <https://www.zakonyprolidi.cz/cs/2006-262>
2. MINISTERSTVO PRÁCE A SOCIÁLNÍCH VĚCÍ, 2026. **VIII.9 Evidence pracovní doby**. In: *Příručka pro personální agendu a odměňování zaměstnanců* [online]. [cit. 2026-04-26]. Dostupné z: <https://ppropo.mpsv.cz/VIII9Evidencepracovnidoby>
3. STÁTNÍ ÚŘAD INSPEKCE PRÁCE, bez data. **Rozvrhování pracovní doby**. In: *suip.gov.cz* [online]. [cit. 2026-04-26]. Dostupné z: https://suip.gov.cz/vsechny-clanky/-/asset_publisher/BwIpyjT9IXe0/content/rozvrhovani-pracovni-do-1
4. ČESKO, 1998. **Zákon č. 111/1998 Sb., o vysokých školách**. In: *Zákony pro lidi* [online]. [cit. 2026-04-26]. Dostupné z: <https://www.zakonyprolidi.cz/cs/1998-111>
5. SOUDNÍ DVŮR EVROPSKÉ UNIE, 2019. **Member States must require employers to set up a system enabling the duration of daily working time to be measured**. In: *curia.europa.eu* [online]. 14. 5. 2019 [cit. 2026-04-26]. Dostupné z: <https://curia.europa.eu/jcms/upload/docs/application/pdf/2019-05/cp190061en.pdf>
6. EUROFOUND, 2020. **Regulations to address work–life balance in digital flexible working arrangements**. In: *eurofound.europa.eu* [online]. [cit. 2026-04-26]. Dostupné z: <https://www.eurofound.europa.eu/en/publications/all/regulations-address-work-life-balance-digital-flexible-working-arrangements>
7. EUROSTAT, bez data. **Job autonomy and pressure at work – statistics**. In: *Statistics Explained* [online]. [cit. 2026-04-26]. Dostupné z: https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Job_autonomy_and_pressure_at_work_-_statistics
8. IBM, bez data. **What Is Requirements Management?** In: *ibm.com* [online]. [cit. 2026-04-26]. Dostupné z: <https://www.ibm.com/think/topics/what-is-requirements-management>
9. ISO 25000, bez data. **ISO/IEC 25010**. In: *iso25000.com* [online]. [cit. 2026-04-26]. Dostupné z: <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>
10. CMU/SEI, 2010. **What Is Your Definition of Software Architecture?** In: *sei.cmu.edu* [online]. Carnegie Mellon University, 2010 [cit. 2026-04-26]. Dostupné z: <https://www.sei.cmu.edu/library/what-is-your-definition-of-software-architecture/>

11. AZURE ARCHITECTURE CENTER, 2025. **Architecture styles – přehled**. In: *learn.microsoft.com* [online]. Microsoft, 2025 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/cs-cz/azure/architecture/guide/architecture-styles/>
12. AVALONIA UI, 2025. **The MVVM Pattern**. In: *docs.avaloniaui.net* [online]. Avalonia, 2025 [cit. 2026-04-26]. Dostupné z: <https://docs.avaloniaui.net/docs/concepts/the-mvvm-pattern/>
13. REACTIVEUI COMMUNITY, 2025. **The Zen of the ViewModel**. In: *reactiveui.net* [online]. ReactiveUI, 2025 [cit. 2026-04-26]. Dostupné z: <https://www.reactiveui.net/docs/handbook/view-models/>
14. MICROSOFT, 2023. **ASP.NET MVC Overview – Model–View–Controller**. In: *learn.microsoft.com* [online]. Microsoft, 2023 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/overview/aspnet-mvc-overview>
15. MICROSOFT, 2023. **Common web application architectures – monolitická aplikace**. In: *learn.microsoft.com* [online]. Microsoft, 2023 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>
16. FOWLER, Martin, 2015. **Monolith First**. In: *martinfowler.com* [online]. 2015 [cit. 2026-04-26]. Dostupné z: <https://martinfowler.com/bliki/MonolithFirst.html>
17. MICROSOFT, 2025. **Microservices architecture style – přehled a trade-offy**. In: *learn.microsoft.com* [online]. Microsoft, 2025 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>
18. MICROSOFT, 2025. **.NET – Build modern apps and services**. In: *dotnet.microsoft.com* [online]. Microsoft, 2025 [cit. 2026-04-26]. Dostupné z: <https://dotnet.microsoft.com/en-us/>
19. MICROSOFT, 2025. **A tour of the C# language**. In: *learn.microsoft.com* [online]. Microsoft, 2025 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/csharp/tour-of-csharp/overview>
20. MICROSOFT, 2024. **Entity Framework Core – SQLite provider**. In: *learn.microsoft.com* [online]. Microsoft, 2024 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/cs-cz/ef/core/providers/sqlite/>
21. SQLITE, 2024. **SQL As Understood By SQLite**. In: *sqlite.org* [online]. SQLite, 2024 [cit. 2026-04-26]. Dostupné z: <https://www.sqlite.org/lang.html>

22. REACTIVEUI COMMUNITY, 2025. **Getting Started**. In: *reactiveui.net* [online].
ReactiveUI, 2025 [cit. 2026-04-26]. Dostupné z: <https://www.reactiveui.net/docs/getting-started/>
23. AVALONIA COMMUNITY, 2024. **Behaviors v Avalonia UI**. In: *docs.avaloniaui.net* [online]. Avalonia Community, 2024 [cit. 2026-04-26]. Dostupné z: <https://docs.avaloniaui.net/docs/controls/behaviors>
24. QUESTPDF, 2025. **QuestPDF – C# PDF Generation Library**. In: *questpdf.com* [online].
QuestPDF, 2025 [cit. 2026-04-26]. Dostupné z: <https://www.questpdf.com/>
25. ARTIFEX SOFTWARE, 2025. **Ghostscript – official documentation**. In: *ghostscript.com* [online]. Artifex Software, 2025 [cit. 2026-04-26]. Dostupné z: <https://www.ghostscript.com/>
26. HABJAN, John a kol., 2025. **Ghostscript.NET – .NET wrapper pro Ghostscript**. In: *github.com/jhabjan/Ghostscript.NET* [online]. 2025 [cit. 2026-04-26]. Dostupné z: <https://github.com/jhabjan/Ghostscript.NET>
27. MICROSOFT, 2025. **Visual Studio – Product documentation**. In: *learn.microsoft.com* [online]. Microsoft, 2025 [cit. 2026-04-26]. Dostupné z: <https://learn.microsoft.com/cs-cz/visualstudio/ide/?view=vs-2022>
28. SQLITESTUDIO, 2025. **SQLiteStudio – Official website**. In: *sqlitestudio.pl* [online]. 2025 [cit. 2026-04-26]. Dostupné z: <https://sqlitestudio.pl/>

SEZNAM PŘÍLOH

Příloha A: Zdrojový kód aplikace TeacherScheduleApp.

Příloha B: Adresářová struktura projektu TeacherScheduleApp.

Příloha C: Levý panel hlavní stránky aplikace.

Příloha D: Pravý panel hlavní stránky aplikace.

Příloha E: Denní pohled aplikace.

Příloha F: Týdenní pohled aplikace.

Příloha G: Měsíční pohled aplikace.

Příloha H: Globální nastavení aplikace.

Příloha I: Ukázka výstupu evidence pracovní doby ve formátu PDF.

Příloha J: Celkový průběh algoritmu vyrovnávání hodin.

Příloha J.1: Určení rozsahu a příprava vyrovnání.

Příloha J.2: Zpracování týdnů a dokončení vyrovnání.

Příloha K: Zjednodušené schéma týdenního vyrovnání pracovní doby.

Příloha K.1: Výpočet týdenního salda.

Příloha K.2: Úpravy týdne podle salda.

Příloha L: Kontrola a vytváření obědových přestávek.

Příloha L.1: Základní kontrola dne.

Příloha L.2: Rozhodnutí o vytvoření obědové přestávky.

Příloha L.3: Umístění a uložení obědové přestávky.

Příloha M: Základní průběh importu rozvrhu ze systému STAG.

Příloha N: Uložení importovaných dat a navazující zpracování evidence.

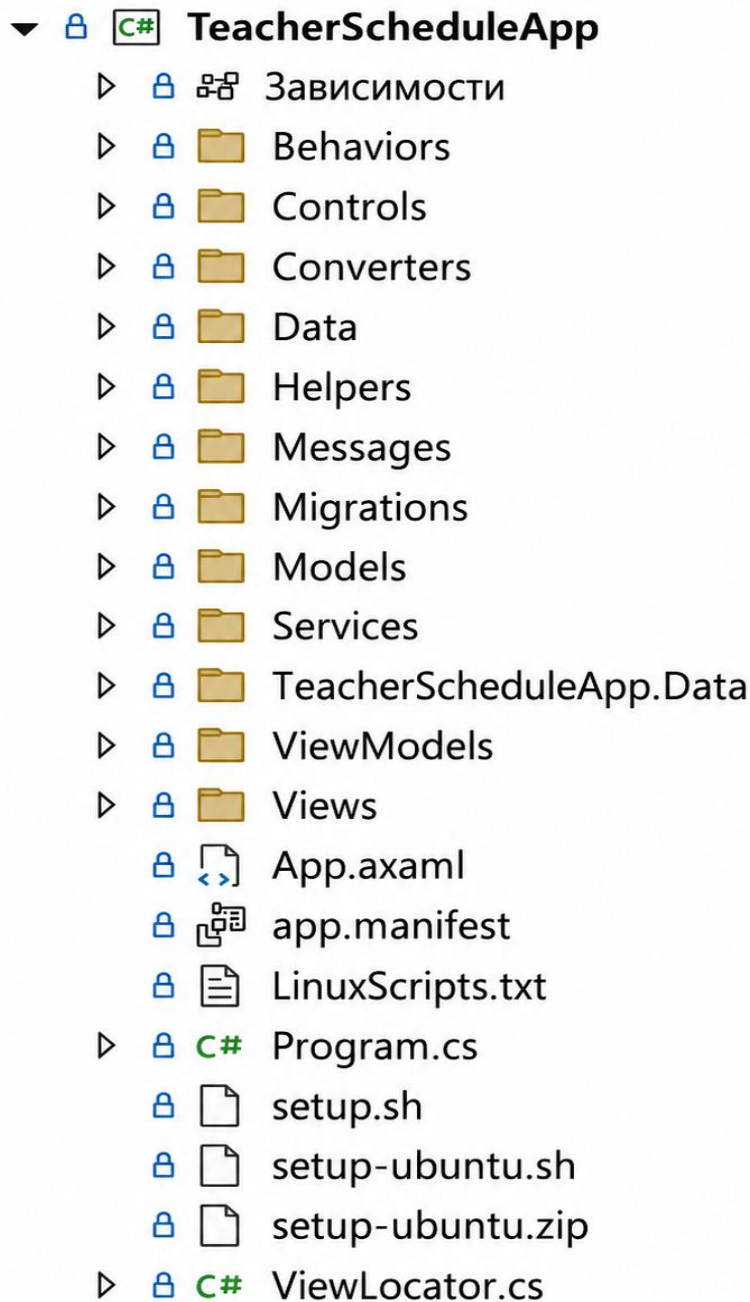
Příloha O: Souhrnná tabulka ověřených scénářů a řešených stavů.

PŘÍLOHA A: Zdrojový kód aplikace TeacherScheduleApp

Tato příloha obsahuje zip archiv s projektem z Visual Studio 2022 jménem TeacherScheduleApp.

PŘÍLOHA B: Adresářová struktura projektu TeacherScheduleApp

Tato příloha obsahuje ukázkou adresářové struktury projektu TeacherScheduleApp ve vývojovém prostředí Visual Studio. Struktura zachycuje hlavní složky a soubory projektu, například Views, ViewModels, Models, Data, Services, Helpers, Controls, Converters, Behaviors a Messages. Přehled slouží k lepší orientaci v rozdělení zdrojového kódu aplikace.



PŘÍLOHA C: Levý panel hlavní stránky aplikace

Tato příloha obsahuje ukázkou levého panelu hlavní stránky aplikace. Levý panel slouží k ovládání aplikace, výběru data, načtení rozvrhu, vytvoření nové události, otevření globálního nastavení a zobrazení náhledu evidence pracovní doby. Součástí panelu je také přehled načtených souborů a nastavení pracovního dne.

Vytvořit událost

Načíst rozvrh

Globální nastavení

EPD náhled

duben 2026

^

v

P	Ú	S	Č	P	S	N
30	31	1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	1	2	3
4	5	6	7	8	9	10

Načtený soubory

^

getRozvrhByUcitel-2025-04-11-14...

11.02.2025 → 07.05.2025

Událostí: 142

Smazat

Rozvrh

v

Nastavení pro den:

14.04.2026

Příchod:

08:30

Odchod:

17:00

Oběd od:

12:30

Oběd do:

13:00

Uložit nastavení

Nastaveno pro den:

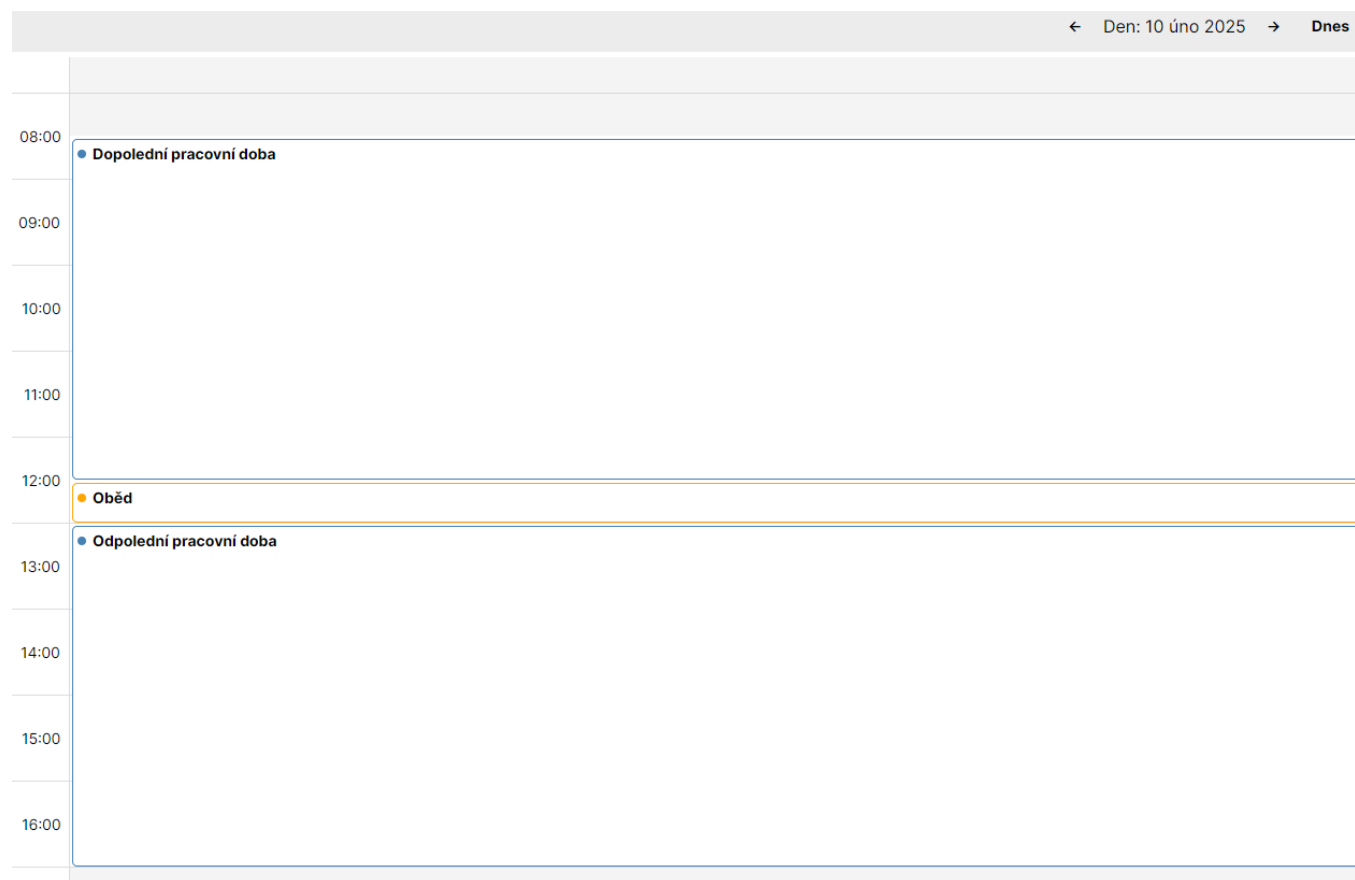
08.00

Skutečně započteno:

08:00 / 08:00

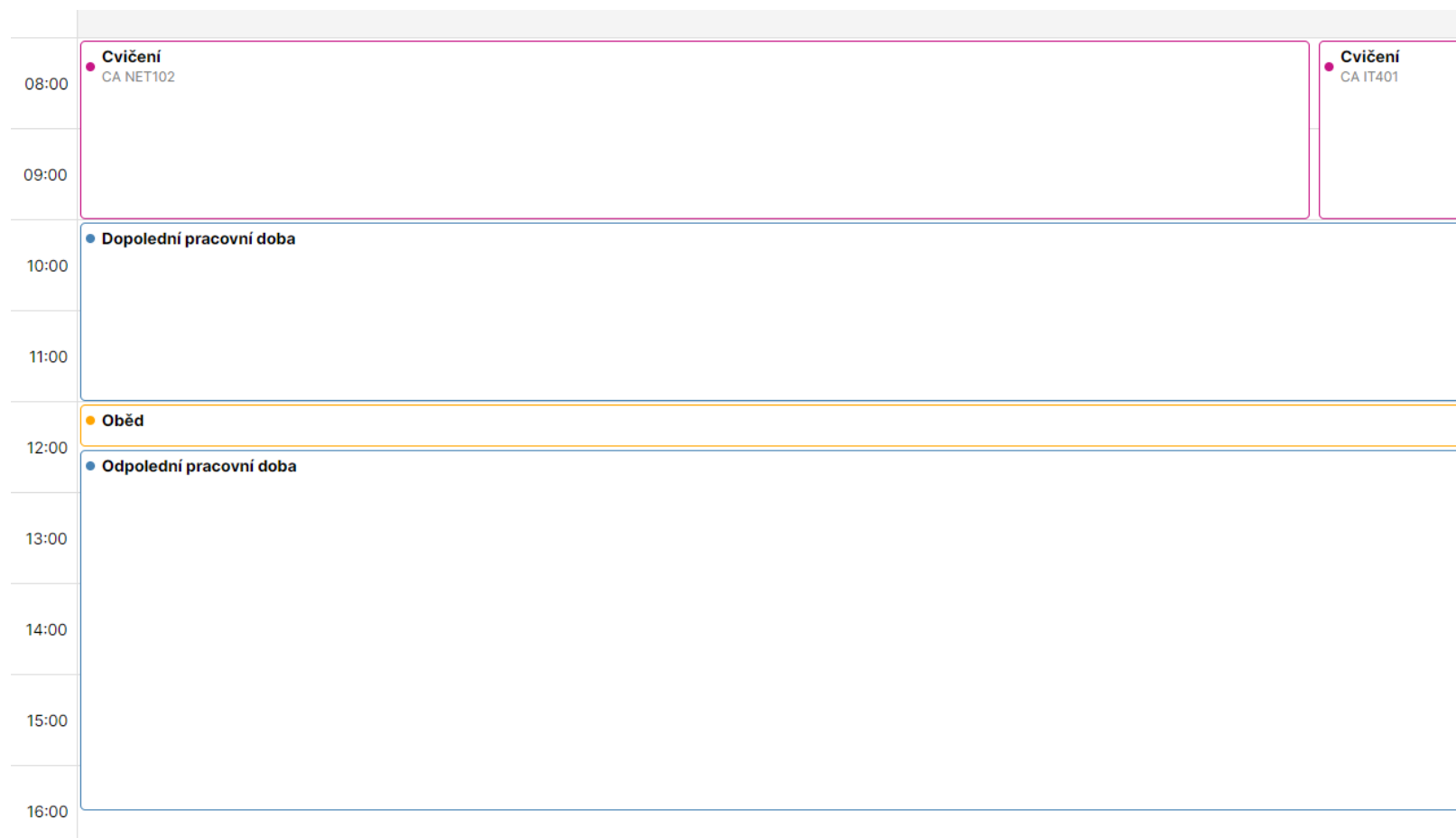
PŘÍLOHA D: Pravý panel hlavní stránky aplikace

Tato příloha obsahuje ukázkou pravé části hlavní stránky aplikace. Pravý panel slouží k zobrazení evidence pracovní doby v kalendářové podobě. Uživatel zde vidí jednotlivé události a pracovní bloky v časovém rozvržení a může kontrolovat jejich umístění v rámci vybraného dne, týdne nebo měsíce.



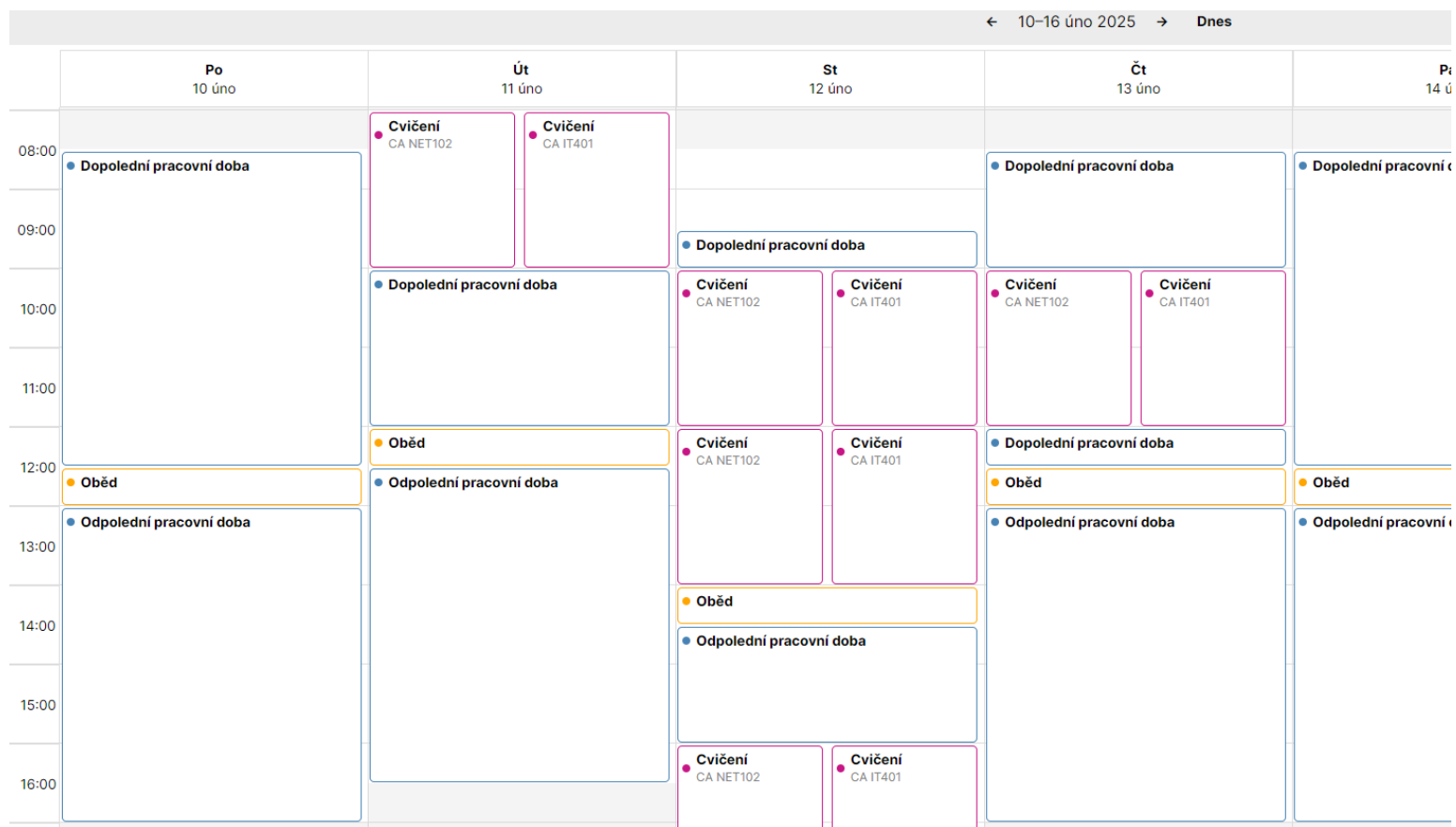
PŘÍLOHA E: Denní pohled aplikace

Tato příloha obsahuje ukázkou denního pohledu aplikace TeacherScheduleApp. Pohled slouží k detailnímu zobrazení událostí a časových bloků v rámci jednoho konkrétního dne.



PŘÍLOHA F: Týdenní pohled aplikace

Tato příloha obsahuje ukázkou týdenního pohledu aplikace. Týdenní pohled umožňuje zobrazit pracovní události v rámci několika po sobě jdoucích dnů a poskytuje přehled o rozložení pracovní doby v týdenním kontextu.



PŘÍLOHA G: Měsíční pohled aplikace

Tato příloha obsahuje ukázkou měsíčního pohledu aplikace. Měsíční pohled poskytuje širší kalendářový přehled o evidovaných událostech a umožňuje orientačně kontrolovat rozložení pracovní doby v rámci celého měsíce.

← únor 2025 → Dnes				
Po	Út	St	Čt	Pá
27	28	29	30	
08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba
3	4	5	6	
08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	08:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba
10	11	12	13	
08:30 Dopolední pracovní doba 12:30 Oběd 13:00 Odpolední pracovní doba	08:00 Cvičení 08:00 Cvičení 10:00 Dopolední pracovní doba 12:00 Oběd 12:30 Odpolední pracovní doba	09:30 Dopolední pracovní doba 10:00 Cvičení 10:00 Cvičení 12:00 Cvičení 12:00 Cvičení 14:00 Oběd	08:30 Dopolední pracovní doba 10:00 Cvičení 10:00 Cvičení 12:00 Dopolední pracovní doba 12:30 Oběd 13:00 Odpolední pracovní doba	08:30 Dopolední pracovní doba 12:30 Oběd 13:00 Odpolední pracovní doba
17	18	19	20	
08:30 Dopolední pracovní doba 12:30 Oběd	08:00 Cvičení 08:00 Cvičení	08:30 ауцкацы 10:00 Cvičení	08:30 Dopolední pracovní doba 10:00 Cvičení	08:30 Dopolední pracovní doba 12:30 Oběd

PŘÍLOHA H: Globální nastavení aplikace

Tato příloha obsahuje ukázkou obrazovky globálního nastavení aplikace. Globální nastavení slouží k úpravě výchozích časových parametrů pracovního dne, přestávek a dalších údajů, které ovlivňují zpracování evidence pracovní doby a vytváření výsledných výstupů.

[← Zpět ke kalendáři](#)

Rok 2025 · Letní semestr

Rok: 2025

☐ Zimní ☒ Letní

Globální pracovní doba · Letní semestr

Od: 08:30

Do: 17:00

Pracovní doba · Letní semestr

Pondělí Úterý Středa Čtvrtek Pátek

Příchod: 08:30

Odchod: 17:00

Oběd, začátek: 12:30

Oběd, konec: 13:00

Další nastavení

Údaje pro PDF

Jméno: Radek Matoušek

Útvar: Katedra informačních technologií

Pauzy

Min. délka pauzy: 00:15

Max. délka pauzy: 01:00

Názvy událostí

Před obědem: Dopolnední pracovní doba

Pro oběd: Oběd

Po obědě: Odpolední pracovní doba

Uložit nastavení

63

PŘÍLOHA I: Ukázka výstupu evidence pracovní doby ve formátu PDF

Tato příloha obsahuje ukázkou výsledného PDF dokumentu evidence pracovní doby. Výstup zobrazuje měsíční přehled pracovní doby, přestávek, odpracovaného času, přesčasů, neodpracované doby a souhrnných údajů za zvolené období.

02-2025

[Uložit PDF](#)

Evidence pracovní doby, včetně přestávek v práci a práce přesčas

Fakulta elektrotechniky a informatiky
jméno: Radek Matoušek
útvár: Katedra informačních technologií

pracovní doba: 08:00–16:30 hod.
docházka za měsíc: únor 2025

Fond prac. doby:
160 hodin

Den	Začátek pracovní doby	1. přestávka		2. přestávka		Konec pracovní doby	Odpracováno	Přesčas	Neodprac.	Poznámka *)
		Od	Do	Od	Do					
1.										
2.										
3.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
4.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
5.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
6.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
7.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
8.										
9.										
10.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
11.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
12.	09:30	14:00	14:30			18:00	08:00:00	00:00:00	00:00:00	
13.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
14.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
15.										
16.										
17.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
18.	12:00					16:00	04:00:00	00:00:00	04:00:00	D
19.							00:00:00	00:00:00	08:00:00	D
20.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
21.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
22.										
23.										
24.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
25.	08:00	12:00	12:30			16:30	08:00:00	00:00:00	00:00:00	
26.	09:30	14:00	14:30			18:00	08:00:00	00:00:00	00:00:00	
27.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
28.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
Celkem							148:00:00	00:00:00	12:00:00	

kontr. č. Σ odprac. - přesčas + neodprac. hodin (odpovídá FPD): 160:00:00

podpis zaměstnance:

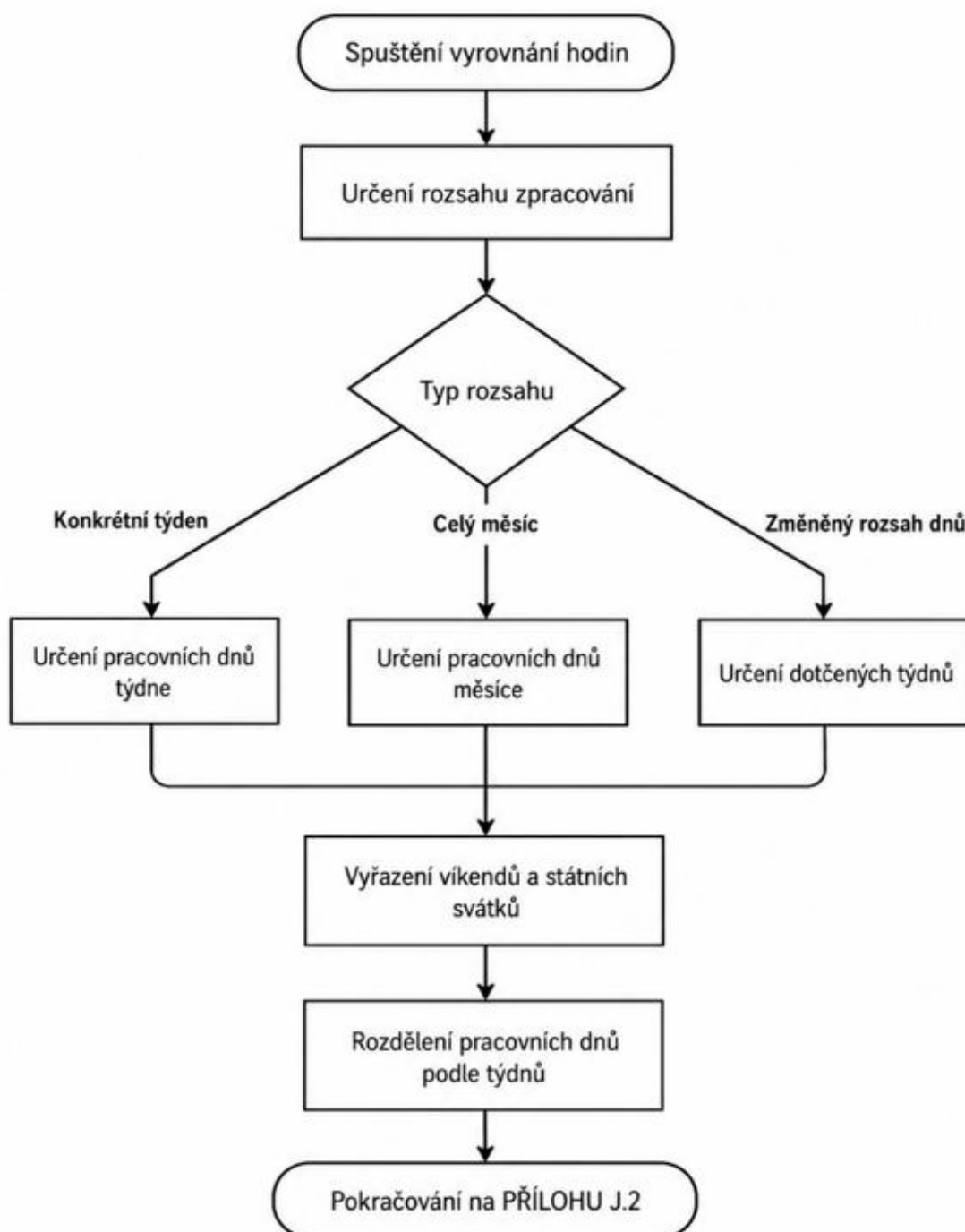
podpis nadřízeného pracovníka:

*) do poznámky: D – dovolená, N – nemoc, OČR – ošetřování, L – lékař, PC – pracovní cesta, S – svátek a ostatní dny pracovního klidu

PŘÍLOHA J: Celkový průběh algoritmu vyrovnávání hodin

Tato příloha obsahuje bloková schémata celkového průběhu algoritmu vyrovnávání hodin. Z důvodu lepší čitelnosti je schéma rozděleno do dvou částí. První část zachycuje určení rozsahu zpracování a přípravu pracovních dnů pro vyrovnání. Druhá část navazuje samotným zpracováním týdnů, kontrolou neúplných týdnů, měsíční normalizací a ukončením vyrovnávání po stabilizaci výsledného stavu evidence.

PŘÍLOHA J.1: Určení rozsahu a příprava vyrovnání.



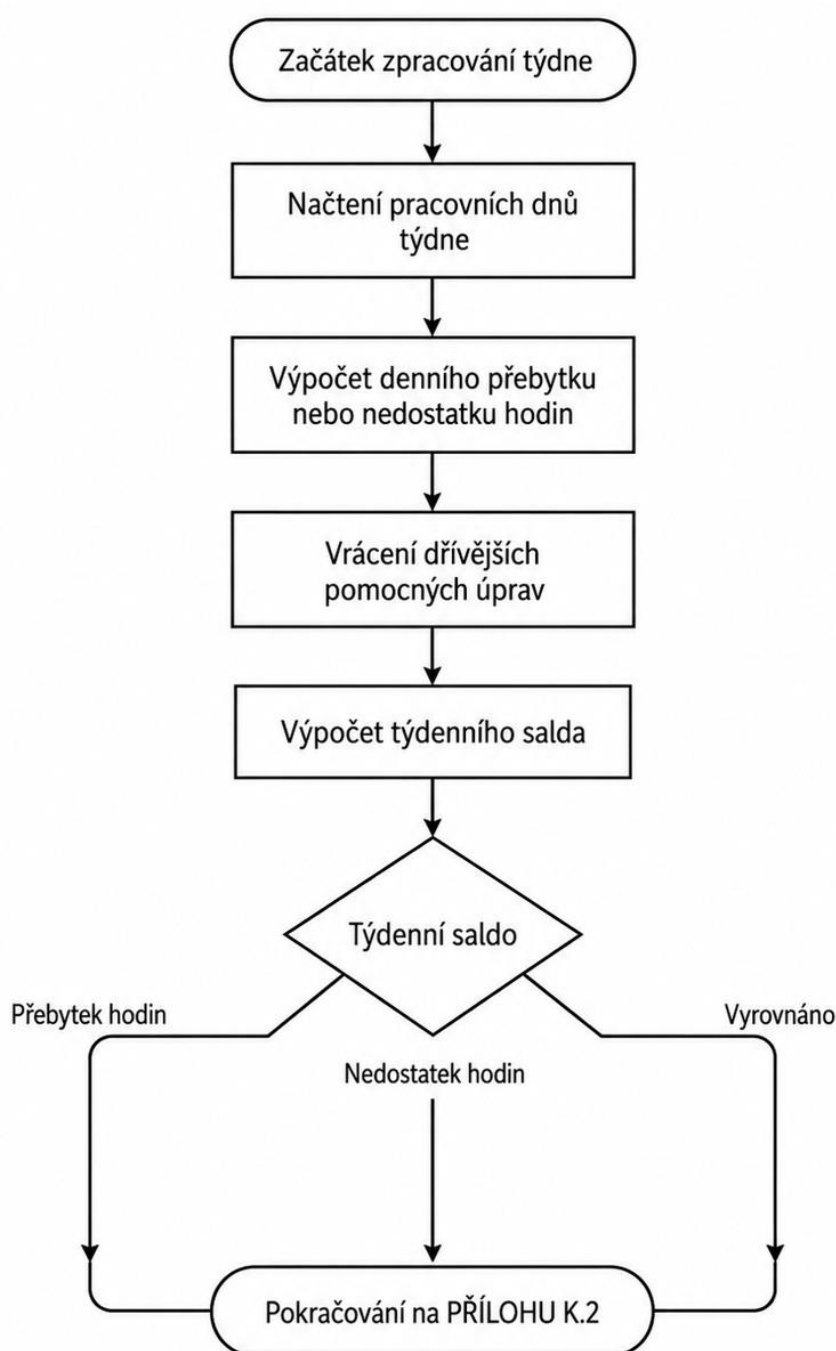
PŘÍLOHA J. 2: Zpracování týdnů a dokončení vyrovnání.



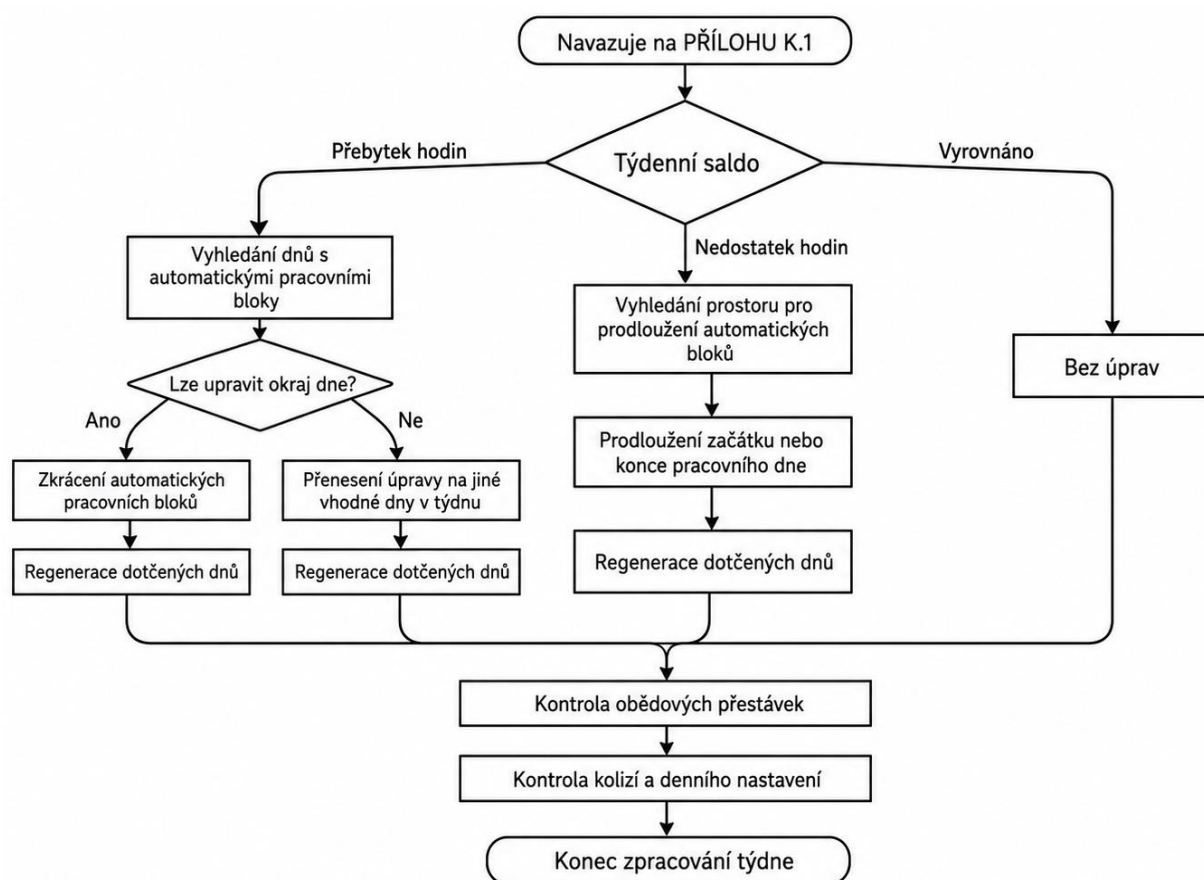
PŘÍLOHA K: Zjednodušené schéma týdenního vyrovnání pracovní doby

Tato příloha obsahuje zjednodušené blokové schéma týdenního vyrovnání pracovní doby. Z důvodu zachování čitelnosti je schéma rozděleno do dvou částí. První část popisuje načtení pracovních dnů týdne, výpočet denních rozdílů a určení týdenního salda. Druhá část znázorňuje postup při přebytku nebo nedostatku hodin, úpravy automatických pracovních bloků a následnou kontrolu obědových přestávek a denního nastavení.

PŘÍLOHA K.1: Výpočet týdenního salda



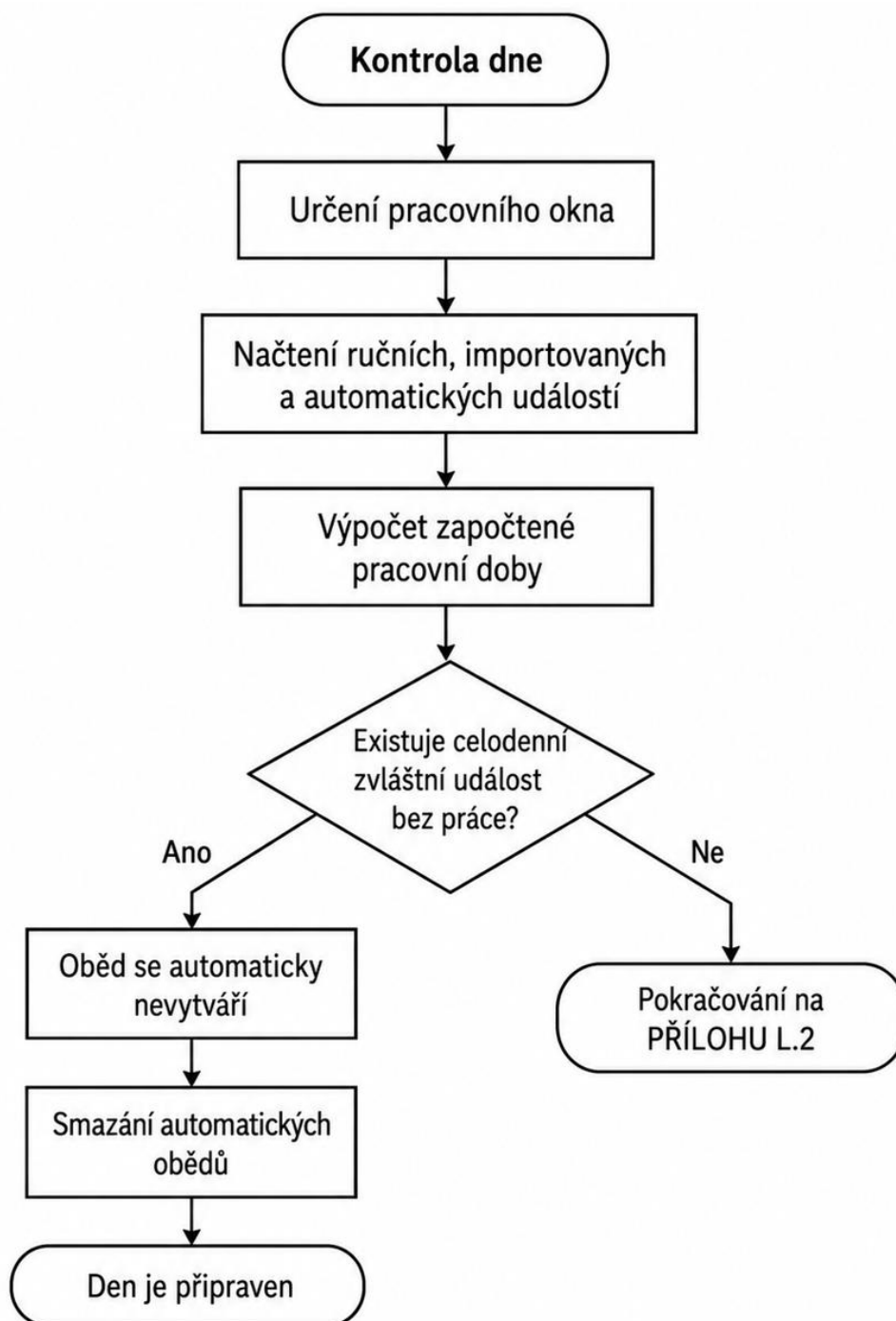
PŘÍLOHA K.2: Úpravy týdne podle salda



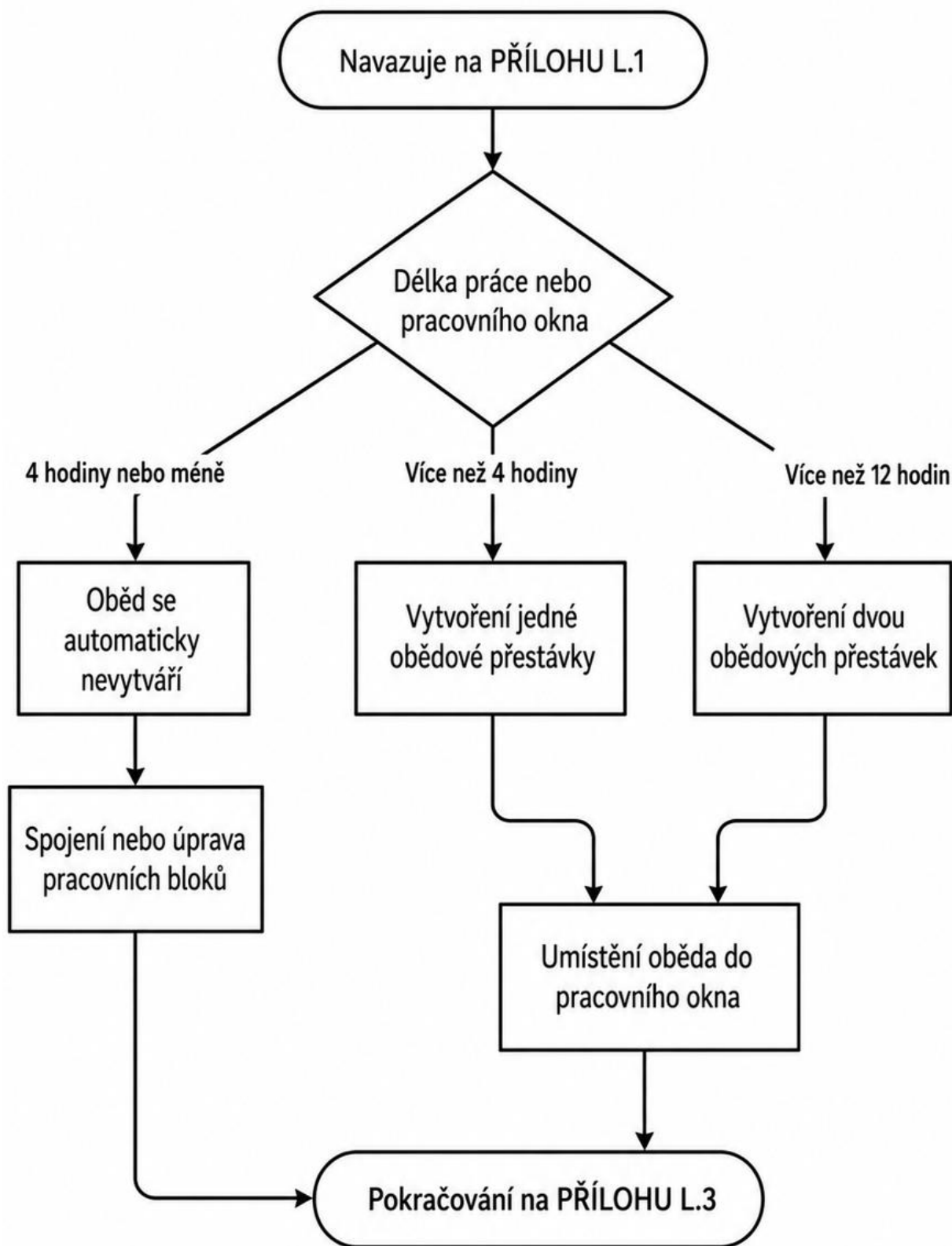
PŘÍLOHA L: Kontrola a vytváření obědových přestávek

Tato příloha obsahuje blokové schéma kontroly a vytváření obědových přestávek. Protože celý postup obsahuje více rozhodovacích kroků, je z důvodu lepší čitelnosti rozdělen do tří částí. První část zachycuje základní kontrolu dne a načtení událostí. Druhá část popisuje rozhodnutí, zda má být obědová přestávka vytvořena podle délky pracovní doby. Třetí část znázorňuje umístění oběda, řešení případné kolize a následné rozdělení pracovních bloků.

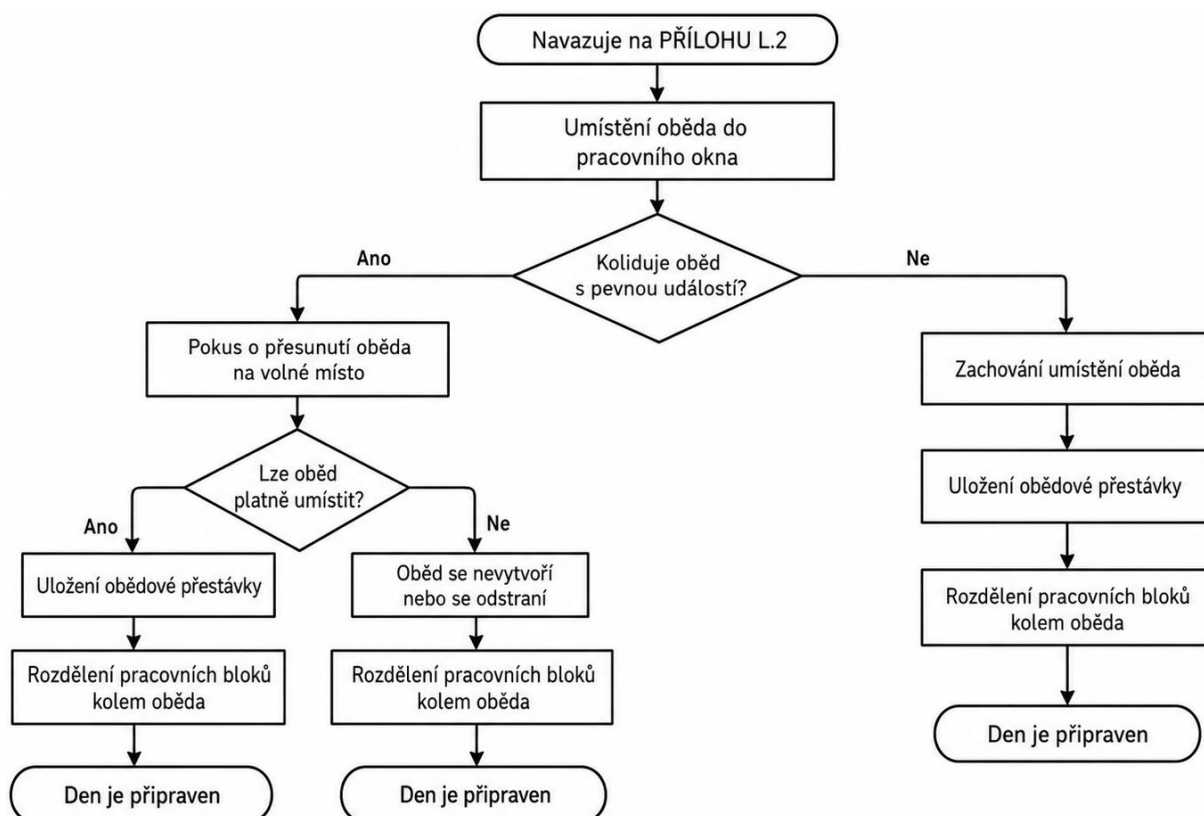
PŘÍLOHA L.1: Základní kontrola dne



PŘÍLOHA L. 2: Rozhodnutí o vytvoření obědové přestávky.

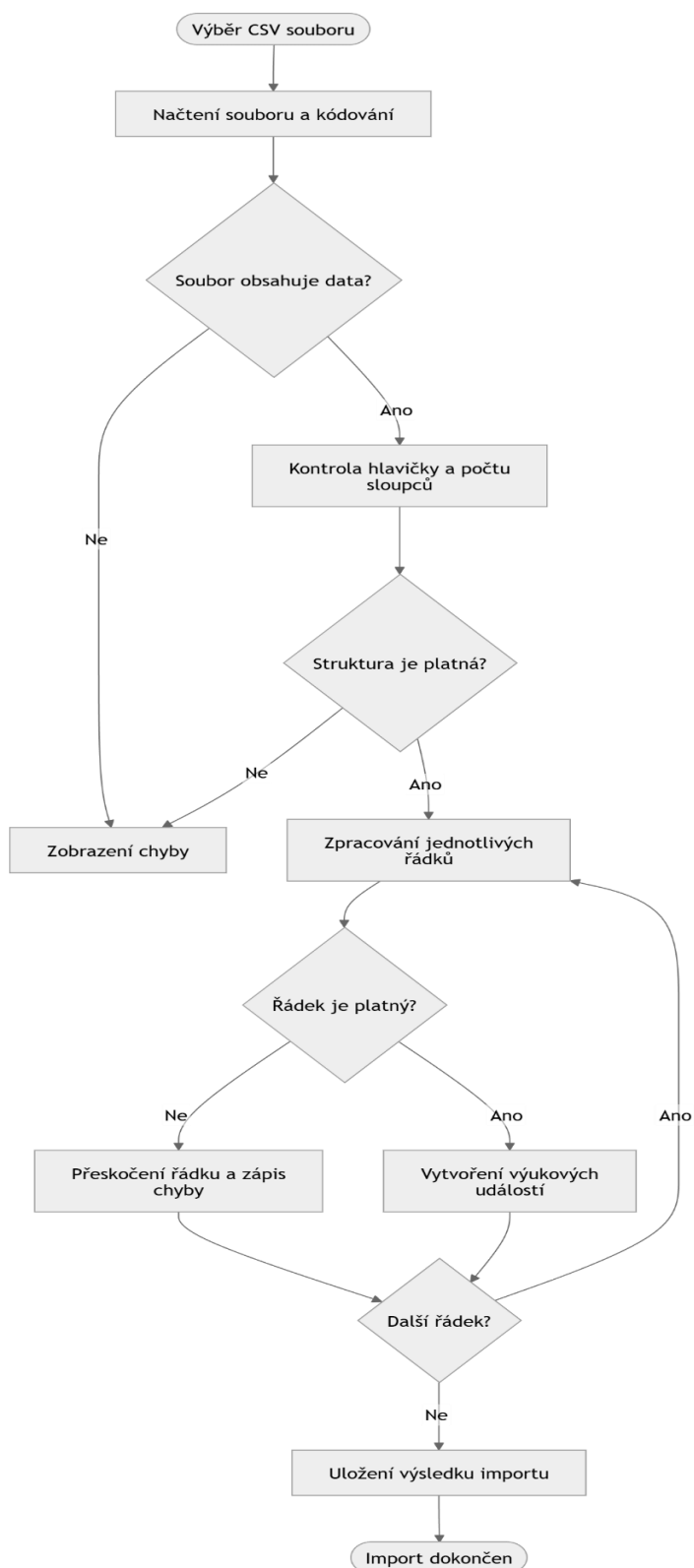


PŘÍLOHA L. 3: Umístění a uložení obědové přestávky.



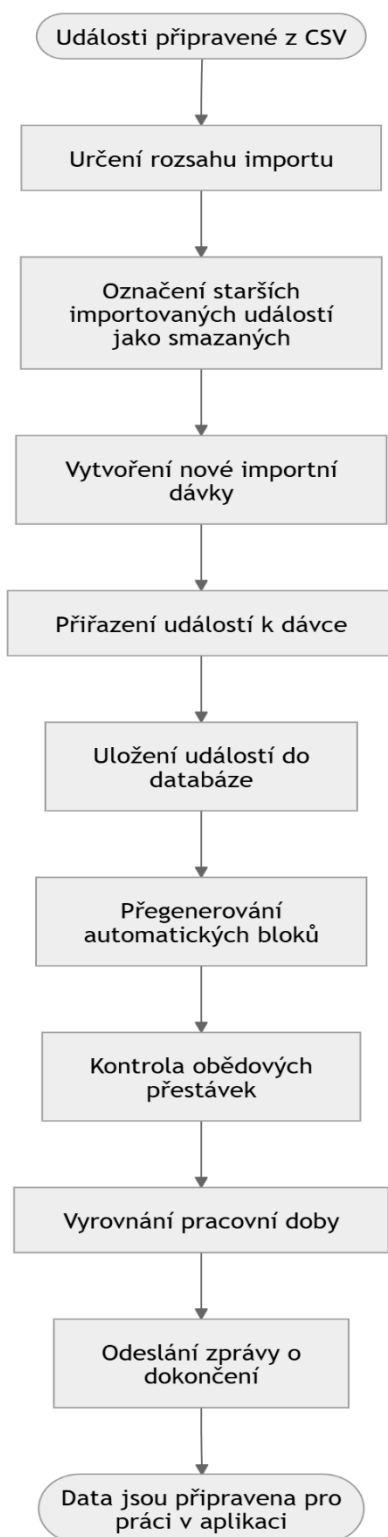
PŘÍLOHA M: Základní průběh importu rozvrhu ze systému STAG

Tato příloha obsahuje blokové schéma základního průběhu importu rozvrhu ze systému STAG. Schéma zachycuje načtení CSV souboru, kontrolu jeho struktury, zpracování jednotlivých řádků a vytvoření výukových událostí.



PŘÍLOHA N: Uložení importovaných dat a navazující zpracování evidence

Tato příloha obsahuje blokové schéma navazujícího zpracování po importu dat. Schéma ukazuje uložení importní dávky, nahrazení starších importovaných záznamů, regeneraci automatických událostí a spuštění vyrovnání pracovní doby.



PŘÍLOHA O: Souhrnná tabulka ověřených scénářů a řešených stavů

Tato příloha obsahuje souhrnnou tabulku ověřených scénářů a řešených stavů. Tabulka zachycuje hlavní testované oblasti aplikace, například import dat, práci s událostmi, obědové přestávky, databázové ukládání, PDF výstup, uživatelské rozhraní a opakovatelnost zpracování.

ID	Oblast	Scénář	Popis problému	Řešení problému
1	Import dat	Import platného CSV souboru ze systému STAG	Je nutné ověřit, zda aplikace umí načíst běžný vstupní soubor a vytvořit z něj události.	Soubor byl načten, události byly vytvořeny podle rozvrhu a uloženy do databáze.
2	Import dat	Import prázdného CSV souboru	Soubor neobsahuje žádná data pro vytvoření událostí.	Import byl odmítnut a uživatel byl informován, že vstupní soubor neobsahuje data.
3	Import dat	CSV soubor s chybějícími nebo nedostatečnými sloupci	Struktura souboru neodpovídá očekávanému formátu.	Aplikace rozpoznala neplatnou strukturu a zabránila uložení nekorektních dat.
4	Import dat	CSV řádek s poškozenou strukturou nebo nepárovými uvozovkami	Jeden řádek nelze bezpečně rozpoznat jako platný záznam.	Problematický řádek byl vyhodnocen jako chybný a nebyl uložen jako pracovní událost.
5	Import dat	CSV řádek s neplatným datem začátku nebo konce období	Z řádku nelze určit platné kalendářní období.	Řádek nebyl použit pro vytvoření událostí a databáze zůstala konzistentní.
6	Import dat	CSV řádek s neplatným časem začátku nebo konce	Z řádku nelze vytvořit platný časový interval.	Řádek byl přeskočen a nebyl uložen jako událost.
7	Import dat	CSV řádek, ve kterém je konec události dříve než začátek	Časový interval události je logicky neplatný.	Aplikace řádek odmítla a nevytvořila chybný záznam.
8	Import dat	CSV řádek s neplatným rozsahem týdnů	Rozvrhový údaj neumožňuje určit týdny, ve kterých má událost vzniknout.	Řádek nebyl použit pro vytvoření událostí.
9	Opakovaný import	Opakovaný import stejného období	Při opakovaném importu by mohly vzniknout duplicitní aktivní události.	Původní importované události v daném rozsahu byly označeny jako neaktivní a nahrazeny novou importní dávkou.
10	Importní dávky	Uložení informací o importované dávce	Po importu je nutné zpětně určit, které události patří ke konkrétnímu importu.	Aplikace vytvořila záznam importní dávky a nově importované události k němu přiřadila.
11	Automatické události	Regenerace automatických pracovních bloků po importu	Po importu se může změnit denní rozvržení práce a volných úseků.	Aplikace přepočítala automaticky generované bloky pro dotčené období.
12	Vyrovnaní pracovní doby	Přepočet evidence po změně rozsahu událostí	Změna událostí může ovlivnit denní, týdenní i měsíční součty.	Aplikace obnovila automatické události a provedla návazné vyrovnaní pracovní doby.
13	Kolize událostí	Překryv dvou ručně zadaných nebo importovaných událostí ve stejném dni	Překrývající se události mohou zkreslit výslednou evidenci.	Události byly označeny jako kolizní, aby je mohl uživatel zkontrolovat.
14	Typy událostí	Zpracování pracovních událostí, výuky a pracovních cest	Různé pracovní typy událostí musí být zahrnuty do výpočtu pracovní doby.	Práce, výuka a pracovní cesta byly při výpočtech zohledněny jako pracovní čas.
15	Typy událostí	Zpracování nepřítomností, například nemoc, dovolená, OČR nebo lékař	Nepřítomnosti musí být odlišeny od běžné práce a správně zobrazeny ve výstupu.	Aplikace rozlišila typ nepřítomnosti a použila jej při evidenci i ve výsledném PDF.

ID	Oblast	Scénář	Popis problému	Řešení problému
16	Denní struktura	Pracovní den delší než 4 hodiny	Při delší pracovní době musí být vytvořena jedna přestávka.	Aplikace vytvořila jednu přestávku a rozdělila pracovní bloky tak, aby se s ní nepřekrývaly.
17	Denní struktura	Pracovní den delší než 12 hodin	Při pracovní době v délce alespoň 12 hodin musí být vytvořeny dvě přestávky.	Aplikace vytvořila dvě přestávky a upravila navazující pracovní bloky podle jejich umístění.
18	Denní struktura	Ruční vytvoření přestávky jako celodenní události	Přestávka nemůže platit pro celý kalendářní den.	Aplikace takový záznam odmítla.
19	Denní struktura	Ruční vytvoření přestávky přesahující jeden kalendářní den	Přestávka musí být evidována pouze v rámci jednoho dne.	Aplikace záznam odmítla.
20	Denní struktura	Ruční vytvoření přestávky s koncem před začátkem	Interval přestávky je časově neplatný.	Aplikace záznam neuložila.
21	Denní struktura	Ruční vytvoření přestávky delší než povolená maximální délka	Délka přestávky překračuje povolené nastavení dne.	Aplikace záznam odmítla nebo uživatele upozornila na překročení délky.
22	Denní struktura	Přestávka kolidující s jinou událostí	Přestávka se může překrývat s výukou nebo jinou ručně zadanou událostí.	Aplikace se pokusila přestávku přesunout do volného místa nebo označila kolizní stav.
23	Nastavení dne	Ruční úprava začátku nebo konce pracovního dne	Uživatel může změnit denní hranice a tím ovlivnit automatické bloky.	Aplikace uložila ruční nastavení a použila ho při dalším zpracování.
24	Nastavení dne	Změna nastavení pro konkrétní den	Jednotlivý den se může lišit od obecných semestrálních pravidel.	Aplikace uložila individuální nastavení dne a odlišila ho od výchozích pravidel.
25	Nastavení semestru	Změna výchozí pracovní doby v semestrálním nastavení	Globální nastavení ovlivňuje další generování pracovních dnů.	Nové nastavení bylo použito jako výchozí základ pro další zpracování.
26	Databáze	Uložení událostí, nastavení dne, nastavení semestru a importních dávek	Data musí být zachována i po ukončení aplikace.	Údaje byly uloženy v lokální databázi SQLite a byly dostupné při dalším spuštění.
27	Databáze	Propojení událostí s importní dávkou	U importovaných událostí musí být dohledatelné, z jakého importu pocházejí.	Údlosti byly propojeny se záznamem importní dávky.
28	Databáze	Práce s lokálním uživatelským záznamem	Evidence je vedena pro lokální profil bez samostatné registrace více uživatelů.	Aplikace pracovala s lokálním profilem osoby, pro kterou je evidence vedena.
29	PDF výstup	Generování měsíčního PDF přehledu	Uživatel potřebuje vytvořit výstup za zvolený měsíc.	Aplikace vytvořila PDF dokument s údaji za zvolený měsíc.
30	PDF výstup	Zobrazení typů nepřítomností a pracovních cest ve výstupu	Ve výstupu musí být jasně rozlišeny jednotlivé typy událostí.	PDF použilo odpovídající zkratky pro dovolenou, nemoc, OČR, lékaře, pracovní cestu a svátek.
31	PDF výstup	Kontrola součtů ve vygenerovaném přehledu	Chybné součty by vedly k nesprávné evidenci pracovní doby.	Součty ve výstupu odpovídaly zpracovaným událostem a pravidlům evidence.
32	Uživatelské rozhraní	Vytvoření nové události přes dialogové okno	Uživatel musí mít možnost ručně doplnit událost do evidence.	Údlost byla vytvořena se zadaným názvem, typem, datem a časem.
33	Uživatelské rozhraní	Úprava existující události přes dialogové okno	Změny události se musí projevit v zobrazení i ve výpočtech.	Upravené údaje byly uloženy a promítny se do kalendáře i výpočtů.
34	Uživatelské rozhraní	Odstranění existující události	Smazaná událost se nesmí dále používat v běžném zobrazení a výpočtech.	Údlost byla odstraněna nebo označena jako smazaná.

ID	Oblast	Scénář	Popis problému	Řešení problému
35	Kalendářové zobrazení	Zobrazení událostí v denním pohledu	Uživatel potřebuje vidět detail jednoho dne.	Události byly zobrazeny podle svého časového rozsahu v rámci konkrétního dne.
36	Kalendářové zobrazení	Zobrazení událostí v týdenním pohledu	Uživatel potřebuje kontrolovat rozložení událostí v rámci týdne.	Události byly zobrazeny v týdenním kontextu.
37	Kalendářové zobrazení	Zobrazení událostí v měsíčním pohledu	Uživatel potřebuje širší přehled o evidenci v měsíci.	Události byly zobrazeny v měsíčním pohledu.
38	Chybové stavy	Pokus o zpracování neplatných vstupních dat	Neplatná data by mohla způsobit nekonzistentní stav evidence.	Aplikace neuložila nekonzistentní záznamy a informovala uživatele o problému.
39	Opakovatelnost zpracování	Opakované provedení stejného importu a následného zpracování	Opakovaný postup musí vést ke stejnému výsledku bez duplicit.	Aplikace vytvořila stejný výsledný stav evidence bez duplicitních aktivních importovaných událostí.